SNN HARWARE ACCELERATOR IN ZYNZ-7020 USING VERILOG

B.PURUSOTH VISAAL

Table of Contents

1. LIF Neuron Core & Layer Manager

Defines the core Leaky Integrate-and-Fire (LIF) neuron using Accumulate-only (AC) synaptic updates in place of traditional Multiply-Accumulate (MAC) operations, together with the layer manager that sequences spike propagation across multiple SNN layers.

Module: lif_neuron_ac

Single LIF neuron with AC-based (multiplier-free) synaptic integration, shift-based leak, and refractory-period control.

```
module lif_neuron_ac #(
parameter NEURON_ID = 0,
parameter DATA_WIDTH = 16, // Q8.8 fixed-point membrane potential
parameter WEIGHT_WIDTH = 8, // INT8 weights
parameter THRESHOLD_WIDTH = 16, // Q8.8 threshold
parameter LEAK_SHIFT = 4, // Leak = vmem >> LEAK_SHIFT (no multiply!)
parameter REFRAC_CYCLES = 5 // Refractory period in cycles
)(
input wire clk,
input wire rst_n,
input wire enable,



//============================================================================
// AC-based Synaptic Input Interface
// Key: spike_valid indicates binary spike (1=spike occurred)
// When spike_valid=1, we ADD weight to membrane (AC operation)
// When spike_valid=0, NO operation needed (energy saved via sparsity)
//============================================================================
input wire spike_valid, // Binary spike input
input wire signed [WEIGHT_WIDTH-1:0] weight, // Pre-loaded weight
input wire is_excitatory, // 1: add, 0: subtract



// Configurable parameters
input wire [THRESHOLD_WIDTH-1:0] threshold,
input wire [DATA_WIDTH-1:0] reset_potential,



// Outputs
output reg spike_out,
output reg [DATA_WIDTH-1:0] membrane_potential,
output wire is_refractory,



// Statistics for energy monitoring
output reg [31:0] ac_operation_count, // Number of AC ops
output reg [31:0] spike_skip_count // Skipped due to sparsity
);



//============================================================================
// Internal State
//============================================================================
reg signed [DATA_WIDTH-1:0] v_mem; // Membrane potential (signed)
reg [3:0] refrac_counter; // Refractory counter



assign is_refractory = (refrac_counter > 0);



//============================================================================
// AC-based Synaptic Integration (NO MULTIPLY!)
//
// Traditional ANN: output = sum(input[i] * weight[i]) -- MAC operations
// Our SNN: output = sum(weight[i]) where spike[i]=1 -- AC only!
//
// Energy benefit: Eliminate ~3.7pJ per operation (multiply cost)
//============================================================================
wire signed [DATA_WIDTH-1:0] weight_extended;
wire signed [DATA_WIDTH:0] v_mem_after_spike;
wire signed [DATA_WIDTH:0] v_mem_after_leak;
```

```
// Sign-extend weight to membrane potential width
assign weight_extended =
{{(DATA_WIDTH-WEIGHT_WIDTH){weight[WEIGHT_WIDTH-1]}}, weight};



// AC operation: Add or subtract weight based on synapse type
// This is the CORE energy savings - no multiply needed!
assign v_mem_after_spike = is_excitatory ?
(v_mem + weight_extended) : // Excitatory: ADD
(v_mem - weight_extended); // Inhibitory: SUBTRACT



//========================================================================
// Leak Implementation using SHIFT (NO MULTIPLY!)
//
// Traditional: v_mem = v_mem * decay_factor
// Our design: v_mem = v_mem - (v_mem >> LEAK_SHIFT)
//
// Approximation: decay ≈ 1 - 2^(-LEAK_SHIFT)
// LEAK_SHIFT=4: decay ≈ 0.9375
// LEAK_SHIFT=3: decay ≈ 0.875
// LEAK_SHIFT=5: decay ≈ 0.96875
//========================================================================
wire signed [DATA_WIDTH-1:0] leak_amount;
assign leak_amount = v_mem >>> LEAK_SHIFT; // Arithmetic right shift
assign v_mem_after_leak = v_mem - leak_amount;



//========================================================================
// Combinational: compute next membrane potential (for immediate spike check)
//========================================================================
wire signed [DATA_WIDTH-1:0] v_mem_next;
wire v_mem_next_valid;



// Saturated synaptic result
wire signed [DATA_WIDTH-1:0] v_mem_spike_sat;
assign v_mem_spike_sat = (v_mem_after_spike[DATA_WIDTH] !=
v_mem_after_spike[DATA_WIDTH-1]) ?
(v_mem_after_spike[DATA_WIDTH] ? {1'b1, {(DATA_WIDTH-1){1'b0}}} :
{1'b0, {(DATA_WIDTH-1){1'b1}}}) :
v_mem_after_spike[DATA_WIDTH-1:0];



// Saturated leak result
wire signed [DATA_WIDTH-1:0] v_mem_leak_sat;
assign v_mem_leak_sat = (v_mem_after_leak[DATA_WIDTH] !=
v_mem_after_leak[DATA_WIDTH-1]) ?
{DATA_WIDTH{1'b0}} :
v_mem_after_leak[DATA_WIDTH-1:0];



// Select next membrane based on spike_valid (same as lif_neuron.v pattern)
assign v_mem_next = spike_valid ? v_mem_spike_sat : v_mem_leak_sat;



// Spike condition: compare v_mem_next (immediate, not 1-cycle delayed)
wire spike_condition = ($signed(v_mem_next) >= $signed(threshold)) && (refrac_counter
== 0);



//========================================================================
// Main Neuron Dynamics
//========================================================================
always @(posedge clk or negedge rst_n) begin
if (!rst_n) begin
v_mem <= 0;
refrac_counter <= 0;
spike_out <= 1'b0;
membrane_potential <= 0;
ac_operation_count <= 0;
spike_skip_count <= 0;



end else if (enable) begin
spike_out <= 1'b0; // Default: no output spike
```

```
if (refrac_counter > 0) begin
//============================================================
// Refractory Period: No integration, just count down
//============================================================
refrac_counter <= refrac_counter - 1;
v_mem <= reset_potential;



end else begin
//============================================================
// Active State: Process incoming spikes
//============================================================



// Update statistics
if (spike_valid) begin
ac_operation_count <= ac_operation_count + 1;
end else begin
spike_skip_count <= spike_skip_count + 1;
end



//============================================================
// Spike Generation (immediate, using v_mem_next)
//============================================================
if (spike_condition) begin
spike_out <= 1'b1;
refrac_counter <= REFRAC_CYCLES;
v_mem <= reset_potential;
end  else begin
v_mem <= v_mem_next;
end
end



// Update output
membrane_potential <= spike_condition ? reset_potential :
(refrac_counter > 0) ? reset_potential : v_mem_next;
end
end



endmodule



SNN LAYER MANAGER
```

Module: snn_layer_manager

Coordinates spike flow and configuration across multiple SNN layers.

```
module snn_layer_manager #(
parameter MAX_LAYERS = 16, // Maximum number of layers
parameter DATA_WIDTH = 48, // Spike data width
parameter CONFIG_WIDTH = 32, // Configuration width
parameter WEIGHT_WIDTH = 8, // Weight precision
parameter VMEM_WIDTH = 16 // Membrane potential precision
)(
input wire clk,
input wire reset,
input wire enable,



// Input interface (from previous layer or input encoder)
input wire [DATA_WIDTH-1:0] s_axis_input_tdata,
input wire s_axis_input_tvalid,
output wire s_axis_input_tready,
input wire s_axis_input_tlast,



// Output interface (to next layer or output decoder)
output wire [DATA_WIDTH-1:0] m_axis_output_tdata,
output wire m_axis_output_tvalid,
input wire m_axis_output_tready,
output wire m_axis_output_tlast,
```

```
// Configuration interface
input wire [7:0] config_layer_id,
input wire [7:0] config_layer_type, // 0: Conv2d, 1: AvgPool2d, 2: MaxPool2d, 3: Linear
input wire [CONFIG_WIDTH-1:0] config_data,
input wire config_write,



// Weight loading interface
input wire [7:0] weight_layer_id,
input wire [15:0] weight_addr,
input wire [WEIGHT_WIDTH-1:0] weight_data,
input wire weight_write,



// Layer execution control
input wire [7:0] execute_layer_id,
input wire execute_start,
output wire execute_done,



// Status and debug
output wire [31:0] total_input_spikes,
output wire [31:0] total_output_spikes,
output wire [MAX_LAYERS-1:0] layer_active_status,
output wire [7:0] current_layer_id
);



// Layer type definitions
localparam LAYER_CONV1D = 4'h0;
localparam LAYER_CONV2D = 4'h1;
localparam LAYER_AVGPOOL2D = 4'h2;
localparam LAYER_MAXPOOL2D = 4'h3;
localparam LAYER_DENSE = 4'h4;
localparam LAYER_INACTIVE = 4'hF;



// Layer configuration storage
reg [7:0] layer_types [0:MAX_LAYERS-1];
reg [CONFIG_WIDTH-1:0] layer_configs [0:MAX_LAYERS-1][0:15]; // 16 config words per
layer
reg layer_enabled [0:MAX_LAYERS-1];



// Current execution state
reg [7:0] active_layer;
reg execution_active;
reg [2:0] exec_state;



// Inter-layer connection signals
wire [DATA_WIDTH-1:0] layer_input_tdata [0:MAX_LAYERS-1];
wire layer_input_tvalid [0:MAX_LAYERS-1];
wire layer_input_tready [0:MAX_LAYERS-1];
wire layer_input_tlast [0:MAX_LAYERS-1];



wire [DATA_WIDTH-1:0] layer_output_tdata [0:MAX_LAYERS-1];
wire layer_output_tvalid [0:MAX_LAYERS-1];
wire layer_output_tready [0:MAX_LAYERS-1];
wire layer_output_tlast [0:MAX_LAYERS-1];



// Layer-specific control signals
wire [31:0] layer_input_spike_count [0:MAX_LAYERS-1];
wire [31:0] layer_output_spike_count [0:MAX_LAYERS-1];
wire layer_computation_done [0:MAX_LAYERS-1];



// Aggregated statistics
reg [31:0] total_input_count;
reg [31:0] total_output_count;
```

```
// Loop variables (Verilog-2001 compatible)
integer cfg_i, cfg_j;



// Configuration management
always @(posedge clk) begin
if (reset) begin
for (cfg_i = 0; cfg_i < MAX_LAYERS; cfg_i = cfg_i + 1) begin
layer_types[cfg_i] <= LAYER_INACTIVE;
layer_enabled[cfg_i] <= 1'b0;
for (cfg_j = 0; cfg_j < 16; cfg_j = cfg_j + 1) begin
layer_configs[cfg_i][cfg_j] <= 32'b0;
end
end
active_layer <= 8'hFF;
execution_active <= 1'b0;
exec_state <= 3'b000;
total_input_count <= 32'b0;
total_output_count <= 32'b0;
end else begin
// Configuration updates
if (config_write && config_layer_id < MAX_LAYERS) begin
if (config_data[31:24] == 8'hFF) begin
// Layer type configuration
layer_types[config_layer_id] <= config_data[7:0];
layer_enabled[config_layer_id] <= 1'b1;
end else begin
// Layer parameter configuration
layer_configs[config_layer_id][config_data[31:28]] <= config_data;
end
end



// Execution control
if (execute_start && !execution_active) begin
active_layer <= execute_layer_id;
execution_active <= 1'b1;
exec_state <= 3'b001;
end else if (execution_active && layer_computation_done[active_layer]) begin
execution_active <= 1'b0;
exec_state <= 3'b000;
end



// Update statistics
total_input_count <= total_input_count + (s_axis_input_tvalid && s_axis_input_tready ? 1 :
0);
total_output_count <= total_output_count + (m_axis_output_tvalid &&
m_axis_output_tready ? 1 : 0);
end
end
//============================================================================
// Layer Pipeline Architecture
//============================================================================
// Design approach: Instead of dynamic layer type selection at runtime,
// we use a fixed pipeline with enable signals controlled by layer_enabled.
// Each slot can be configured with parameters, but the layer type is
// determined by the pipeline position (compile-time).
//
// For runtime layer type selection, use a crossbar switch architecture
// with pre-instantiated layer processors.
//============================================================================



// Internal pipeline signals
wire [DATA_WIDTH-1:0] pipe_data [0:MAX_LAYERS];
wire pipe_valid [0:MAX_LAYERS];
wire pipe_ready [0:MAX_LAYERS];
wire pipe_last [0:MAX_LAYERS];



// Connect input to pipeline start
assign pipe_data[0] = s_axis_input_tdata;
assign pipe_valid[0] = s_axis_input_tvalid;
assign pipe_last[0] = s_axis_input_tlast;
assign s_axis_input_tready = pipe_ready[0];
```

```
// Connect pipeline end to output
assign m_axis_output_tdata = pipe_data[MAX_LAYERS];
assign m_axis_output_tvalid = pipe_valid[MAX_LAYERS];
assign m_axis_output_tlast = pipe_last[MAX_LAYERS];
assign pipe_ready[MAX_LAYERS] = m_axis_output_tready;



//============================================================================
// Layer Pipeline Generation
//============================================================================
// Each layer slot acts as a configurable processing element
// When layer_enabled[i] is 0, the slot passes data through
// When layer_enabled[i] is 1, the slot processes data based on config
//============================================================================
genvar i;
generate
for (i = 0; i < MAX_LAYERS; i = i + 1) begin : layer_pipeline



// Per-layer processing registers
reg [DATA_WIDTH-1:0] proc_data_out;
reg proc_valid_out;
reg proc_last_out;
reg proc_ready_out;



// Layer processing state machine
reg [2:0] layer_state;
localparam L_IDLE = 3'd0;
localparam L_RECEIVE = 3'd1;
localparam L_PROCESS = 3'd2;
localparam L_OUTPUT = 3'd3;
localparam L_DONE = 3'd4;



// Processing buffer
reg [DATA_WIDTH-1:0] data_buffer;
reg data_buffered;
reg last_buffered;



// Computation done flag for this layer
reg layer_done;
assign layer_computation_done[i] = layer_done;



always @(posedge clk) begin
if (reset) begin
layer_state <= L_IDLE;
proc_data_out <= {DATA_WIDTH{1'b0}};
proc_valid_out <= 1'b0;
proc_last_out <= 1'b0;
proc_ready_out <= 1'b1;
data_buffer <= {DATA_WIDTH{1'b0}};
data_buffered <= 1'b0;
last_buffered <= 1'b0;
layer_done <= 1'b0;
end else if (!enable) begin
// Pass through when disabled
proc_data_out <= pipe_data[i];
proc_valid_out <= pipe_valid[i];
proc_last_out <= pipe_last[i];
proc_ready_out <= pipe_ready[i+1];
layer_done <= 1'b1;
end else if (!layer_enabled[i]) begin
// Pass through when layer not configured
proc_data_out <= pipe_data[i];
proc_valid_out <= pipe_valid[i];
proc_last_out <= pipe_last[i];
proc_ready_out <= pipe_ready[i+1];
layer_done <= 1'b1;
end else begin
// Layer is enabled - process based on layer type
layer_done <= 1'b0;



case (layer_state)
L_IDLE: begin
proc_ready_out <= 1'b1;
```

```
proc_valid_out <= 1'b0;
if (pipe_valid[i] && proc_ready_out) begin
data_buffer <= pipe_data[i];
last_buffered <= pipe_last[i];
data_buffered <= 1'b1;
layer_state <= L_PROCESS;
end
end



L_PROCESS: begin
proc_ready_out <= 1'b0;
// Simple processing based on layer type
// This can be extended with actual computation
case (layer_types[i][3:0])
LAYER_CONV1D, LAYER_CONV2D: begin
// Placeholder: pass through (actual conv would be here)
proc_data_out <= data_buffer;
end
LAYER_AVGPOOL2D: begin
// Placeholder: simple scaling (actual pooling would be here)
proc_data_out <= data_buffer;
end
LAYER_MAXPOOL2D: begin
// Placeholder: pass through (actual max pooling would be here)
proc_data_out <= data_buffer;
end
LAYER_DENSE: begin
// Placeholder: pass through (actual dense would be here)
proc_data_out <= data_buffer;
end
default: begin
proc_data_out <= data_buffer;
end
endcase
proc_last_out <= last_buffered;
layer_state <= L_OUTPUT;
end



L_OUTPUT: begin
proc_valid_out <= 1'b1;
if (pipe_ready[i+1]) begin
proc_valid_out <= 1'b0;
data_buffered <= 1'b0;
if (last_buffered) begin
layer_state <= L_DONE;
end else begin
layer_state <= L_IDLE;
end
end
end



L_DONE: begin
layer_done <= 1'b1;
proc_ready_out <= 1'b0;
proc_valid_out <= 1'b0;
// Wait for new execution command
if (!execution_active) begin
layer_state <= L_IDLE;
end
end



default: layer_state <= L_IDLE;
endcase
end
end



// Connect to pipeline
assign pipe_data[i+1] = proc_data_out;
assign pipe_valid[i+1] = proc_valid_out;
assign pipe_last[i+1] = proc_last_out;
assign pipe_ready[i] = proc_ready_out;



// Statistics tracking
reg [31:0] input_count;
```

```
reg [31:0] output_count;



always @(posedge clk) begin
if (reset) begin
input_count <= 32'b0;
output_count <= 32'b0;
end else begin
if (pipe_valid[i] && pipe_ready[i])
input_count <= input_count + 1;
if (pipe_valid[i+1] && pipe_ready[i+1])
output_count <= output_count + 1;
end
end



assign layer_input_spike_count[i] = input_count;
assign layer_output_spike_count[i] = output_count;



// Unused inter-layer signals - tie off
assign layer_input_tdata[i] = pipe_data[i];
assign layer_input_tvalid[i] = pipe_valid[i];
assign layer_input_tlast[i] = pipe_last[i];
assign layer_input_tready[i] = pipe_ready[i];
assign layer_output_tdata[i] = pipe_data[i+1];
assign layer_output_tvalid[i] = pipe_valid[i+1];
assign layer_output_tlast[i] = pipe_last[i+1];
assign layer_output_tready[i] = pipe_ready[i+1];



end
endgenerate



// Status outputs
assign execute_done = !execution_active;
assign current_layer_id = active_layer;
assign total_input_spikes = total_input_count;
assign total_output_spikes = total_output_count;



// Layer active status - generate layer_active_status from layer_enabled array
genvar k;
generate
for (k = 0; k < MAX_LAYERS; k = k + 1) begin : gen_layer_status
assign layer_active_status[k] = layer_enabled[k];
end
endgenerate



endmodule
```

2. 1D Average Pooling

Implements 1D average pooling over spiking feature streams, reducing temporal/spatial resolution while preserving spike-rate information.

Module: snn_avgpool1d

1D average-pooling datapath for spiking inputs.

```
module snn_avgpool1d #(
parameter INPUT_LENGTH = 128,
parameter INPUT_CHANNELS = 32,
parameter POOL_SIZE = 2,
parameter STRIDE = 2,
parameter OUTPUT_LENGTH = INPUT_LENGTH / STRIDE,
// Precision parameters
parameter VMEM_WIDTH = 16, // Q8.8 membrane potential
parameter ADDR_WIDTH = 16,
parameter TIMESTAMP_WIDTH = 16
)(
input wire clk,
input wire rst_n,
input wire enable,



// Input spike interface (AXI-Stream)
input wire s_axis_input_tvalid,
input wire [31:0] s_axis_input_tdata,
input wire s_axis_input_tlast,
output reg s_axis_input_tready,



// Output spike interface (AXI-Stream)
output reg m_axis_output_tvalid,
output reg [31:0] m_axis_output_tdata,
output reg m_axis_output_tlast,
input wire m_axis_output_tready,



// Configuration
input wire config_valid,
input wire [31:0] config_data,



// Status
output reg busy,
output reg layer_done
);



// Accumulator memory for Verilog-2001 compatibility
reg [VMEM_WIDTH-1:0] accumulator_mem
[0:INPUT_CHANNELS*OUTPUT_LENGTH-1];
reg [7:0] spike_count_mem [0:INPUT_CHANNELS*OUTPUT_LENGTH-1];



// Input spike buffer
reg input_spike_buffer [0:INPUT_LENGTH-1];
reg [7:0] input_channel_id;
reg [15:0] input_position;



// Internal signals
reg [2:0] pool_state;
reg [15:0] current_ch;
reg [15:0] current_pos;
reg [31:0] pool_sum;
reg [7:0] pool_count;
integer k; // Moved outside always block for Verilog-2001



// State machine parameters
localparam IDLE = 3'b000;
localparam PROCESS_SPIKES = 3'b001;
localparam POOLING = 3'b010;
localparam OUTPUT_SPIKES = 3'b011;
```

```
localparam DONE = 3'b100;



// Helper function to calculate flattened index
function [31:0] pool_index;
input [15:0] ch;
input [15:0] pos;
begin
pool_index = ch * OUTPUT_LENGTH + pos;
end
endfunction



// Input spike parsing
always @(*) begin
if (s_axis_input_tvalid && s_axis_input_tready) begin
input_channel_id = s_axis_input_tdata[15:8];
input_position = s_axis_input_tdata[31:16];
end else begin
input_channel_id = 8'b0;
input_position = 16'b0;
end
end



// Main state machine
always @(posedge clk or negedge rst_n) begin
if (!rst_n) begin
pool_state <= IDLE;
current_ch <= 16'b0;
current_pos <= 16'b0;
pool_sum <= 32'b0;
pool_count <= 8'b0;



s_axis_input_tready <= 1'b0;
m_axis_output_tvalid <= 1'b0;
m_axis_output_tdata <= 32'b0;
m_axis_output_tlast <= 1'b0;



busy <= 1'b0;
layer_done <= 1'b0;



end else if (enable) begin
case (pool_state)
IDLE: begin
if (config_valid) begin
pool_state <= PROCESS_SPIKES;
s_axis_input_tready <= 1'b1;
busy <= 1'b1;
end else begin
s_axis_input_tready <= 1'b1;
busy <= 1'b0;
end
end



PROCESS_SPIKES: begin
if (s_axis_input_tvalid && s_axis_input_tready) begin
// Store input spike
if (input_position < INPUT_LENGTH) begin
input_spike_buffer[input_position] <= 1'b1;
end



if (s_axis_input_tlast) begin
pool_state <= POOLING;
s_axis_input_tready <= 1'b0;
current_ch <= 16'b0;
current_pos <= 16'b0;
end
end
end
```

```
POOLING: begin
// Perform 1D average pooling
pool_sum <= 32'b0;
pool_count <= 8'b0;



// Count spikes in pool window
for (k = 0; k < POOL_SIZE; k = k + 1) begin
if ((current_pos * STRIDE + k) < INPUT_LENGTH) begin
if (input_spike_buffer[current_pos * STRIDE + k]) begin
pool_count <= pool_count + 1;
end
end
end



// Store average
accumulator_mem[pool_index(current_ch, current_pos)] <= pool_count;
spike_count_mem[pool_index(current_ch, current_pos)] <= pool_count;



// Move to next position
if (current_pos < OUTPUT_LENGTH - 1) begin
current_pos <= current_pos + 1;
end else begin
current_pos <= 16'b0;
if (current_ch < INPUT_CHANNELS - 1) begin
current_ch <= current_ch + 1;
end else begin
pool_state <= OUTPUT_SPIKES;
current_ch <= 16'b0;
current_pos <= 16'b0;
end
end
end



OUTPUT_SPIKES: begin
if (m_axis_output_tready) begin
// Check if we should output a spike
if (spike_count_mem[pool_index(current_ch, current_pos)] > 0) begin
m_axis_output_tvalid <= 1'b1;
m_axis_output_tdata <= {current_pos, current_ch[7:0], 7'b0, 1'b1};
end else begin
m_axis_output_tvalid <= 1'b0;
end



// Move to next position
if (current_pos < OUTPUT_LENGTH - 1) begin
current_pos <= current_pos + 1;
end else begin
current_pos <= 16'b0;
if (current_ch < INPUT_CHANNELS - 1) begin
current_ch <= current_ch + 1;
end else begin
m_axis_output_tlast <= 1'b1;
pool_state <= DONE;
end
end
end
end



DONE: begin
m_axis_output_tvalid <= 1'b0;
m_axis_output_tlast <= 1'b0;
busy <= 1'b0;
layer_done <= 1'b1;
pool_state <= IDLE;
end



default: begin
pool_state <= IDLE;
end
endcase
end
end
```

```
// Initialize memories
integer i;
initial begin
for (i = 0; i < INPUT_CHANNELS * OUTPUT_LENGTH; i = i + 1) begin
accumulator_mem[i] = 16'b0;
spike_count_mem[i] = 8'b0;
end



for (i = 0; i < INPUT_LENGTH; i = i + 1) begin
input_spike_buffer[i] = 1'b0;
end
end



endmodule
```

3. 2D Average Pooling

Implements 2D average pooling for spiking feature maps, used between convolutional SNN layers to downsample spatial dimensions.

Module: snn_avgpool2d

2D average-pooling datapath for spiking feature maps.

```
module snn_avgpool2d #(
parameter INPUT_WIDTH = 28, // Input feature map width
parameter INPUT_HEIGHT = 28, // Input feature map height
parameter INPUT_CHANNELS = 32, // Number of input channels
parameter POOL_SIZE = 2, // Pooling window size (2x2, 3x3, etc.)
parameter STRIDE = 2, // Pooling stride
parameter VMEM_WIDTH = 16, // Membrane potential precision
parameter POOL_THRESHOLD = 16'h2000 // Pooling threshold (0.125 in Q8.8)
)(
input wire clk,
input wire reset,
input wire enable,



// Input spike interface (AXI-Stream)
input wire [31:0] s_axis_input_tdata, // {channel[7:0], y[7:0], x[7:0], valid[7:0]}
input wire s_axis_input_tvalid,
output reg s_axis_input_tready,
input wire s_axis_input_tlast,



// Output spike interface (AXI-Stream)
output reg [31:0] m_axis_output_tdata, // {channel[7:0], y[7:0], x[7:0], valid[7:0]}
output reg m_axis_output_tvalid,
input wire m_axis_output_tready,
output reg m_axis_output_tlast,



// Configuration
input wire [15:0] threshold_config,
input wire [7:0] decay_factor, // Membrane leak factor
input wire [7:0] pooling_weight, // Weight for each spike in pool



// Status
output reg [31:0] input_spike_count,
output reg [31:0] output_spike_count,
output reg computation_done
);



// Calculated parameters
localparam OUTPUT_WIDTH = (INPUT_WIDTH - POOL_SIZE) / STRIDE + 1;
localparam OUTPUT_HEIGHT = (INPUT_HEIGHT - POOL_SIZE) / STRIDE + 1;



// Internal signals
reg [7:0] input_x, input_y, input_ch;
reg input_spike_valid;



// Pooling accumulator memory (for each output position)
reg signed [VMEM_WIDTH-1:0] pool_accumulator
[0:INPUT_CHANNELS-1][0:OUTPUT_HEIGHT-1][0:OUTPUT_WIDTH-1];



// Spike counting for average calculation
reg [7:0] spike_count
[0:INPUT_CHANNELS-1][0:OUTPUT_HEIGHT-1][0:OUTPUT_WIDTH-1];



// Timing control for pooling windows
reg [15:0] time_window_counter;
reg [15:0] pooling_window_size; // Number of time steps for pooling window
// Output spike generation
reg [7:0] output_x, output_y, output_ch;
```

```
reg output_spike_pending;
reg [2:0] output_state;



// Performance counters
reg [31:0] local_input_count;
reg [31:0] local_output_count;



// Loop variables for reset
integer i, j, k;
integer ch, y, x; // Loop variables for decay



// Pool coordinate calculation registers
reg [7:0] pool_x, pool_y;



// Output calculation registers (moved outside always block for Verilog-2001)
reg signed [VMEM_WIDTH-1:0] avg_value;
reg [7:0] current_spike_count;



// Input spike parsing
always @(*) begin
input_x = s_axis_input_tdata[7:0];
input_y = s_axis_input_tdata[15:8];
input_ch = s_axis_input_tdata[23:16];
input_spike_valid = s_axis_input_tdata[31:24] != 0;
end



// Calculate which pooling window contains the input coordinate
function [7:0] calc_pool_x;
input [7:0] input_x;
begin
calc_pool_x = input_x / STRIDE;
end
endfunction



function [7:0] calc_pool_y;
input [7:0] input_y;
begin
calc_pool_y = input_y / STRIDE;
end
endfunction



// Check if input coordinate contributes to pooling window
function is_in_pool_window;
input [7:0] in_x, in_y, pool_x, pool_y;
reg [7:0] pool_start_x, pool_start_y;
reg [7:0] pool_end_x, pool_end_y;
begin
pool_start_x = pool_x * STRIDE;
pool_start_y = pool_y * STRIDE;
pool_end_x = pool_start_x + POOL_SIZE - 1;
pool_end_y = pool_start_y + POOL_SIZE - 1;



is_in_pool_window = (in_x >= pool_start_x) && (in_x <= pool_end_x) &&
(in_y >= pool_start_y) && (in_y <= pool_end_y);
end
endfunction



// Main processing logic
always @(posedge clk) begin
if (reset) begin
// Reset all state
s_axis_input_tready <= 1'b1;
m_axis_output_tvalid <= 1'b0;
m_axis_output_tlast <= 1'b0;
computation_done <= 1'b0;
output_spike_pending <= 1'b0;
```

```
output_state <= 3'b000;
local_input_count <= 32'b0;
local_output_count <= 32'b0;
time_window_counter <= 16'b0;
pooling_window_size <= 16'd100; // Default: 100 time steps per pooling window



// Reset accumulators and spike counts
for (i = 0; i < INPUT_CHANNELS; i = i + 1) begin
for (j = 0; j < OUTPUT_HEIGHT; j = j + 1) begin
for (k = 0; k < OUTPUT_WIDTH; k = k + 1) begin
pool_accumulator[i][j][k] <= 0;
spike_count[i][j][k] <= 0;
end
end
end



end else if (enable) begin



// Time window management
time_window_counter <= time_window_counter + 1;



// Input spike processing
if (s_axis_input_tvalid && s_axis_input_tready && input_spike_valid) begin
local_input_count <= local_input_count + 1;



// Calculate which pooling windows this spike contributes to
pool_x = calc_pool_x(input_x);
pool_y = calc_pool_y(input_y);



// Check bounds
if (pool_x < OUTPUT_WIDTH && pool_y < OUTPUT_HEIGHT &&
is_in_pool_window(input_x, input_y, pool_x, pool_y)) begin



// Add spike to accumulator
pool_accumulator[input_ch][pool_y][pool_x] <=
pool_accumulator[input_ch][pool_y][pool_x] + pooling_weight;



// Increment spike count for average calculation
spike_count[input_ch][pool_y][pool_x] <=
spike_count[input_ch][pool_y][pool_x] + 1;
end
end



// Pooling window completion and output generation
if (time_window_counter >= pooling_window_size) begin
time_window_counter <= 16'b0;
if (!output_spike_pending) begin
output_state <= 3'b001; // Start output generation
output_ch <= 8'b0;
output_y <= 8'b0;
output_x <= 8'b0;
end
end



// Output spike generation state machine
case (output_state)
3'b001: begin // Check current position for spike
current_spike_count = spike_count[output_ch][output_y][output_x];



if (current_spike_count > 0) begin
// Calculate average: accumulator / spike_count
avg_value = pool_accumulator[output_ch][output_y][output_x] / current_spike_count;
```

```
if (avg_value >= threshold_config) begin
// Generate output spike
output_spike_pending <= 1'b1;
local_output_count <= local_output_count + 1;
end
end



// Reset accumulator for next window
pool_accumulator[output_ch][output_y][output_x] <= 0;
spike_count[output_ch][output_y][output_x] <= 0;



output_state <= 3'b010;
end



3'b010: begin // Move to next position
if (output_x < OUTPUT_WIDTH - 1) begin
output_x <= output_x + 1;
output_state <= 3'b001;
end else if (output_y < OUTPUT_HEIGHT - 1) begin
output_x <= 8'b0;
output_y <= output_y + 1;
output_state <= 3'b001;
end else if (output_ch < INPUT_CHANNELS - 1) begin
output_x <= 8'b0;
output_y <= 8'b0;
output_ch <= output_ch + 1;
output_state <= 3'b001;
end else begin
// Finished all positions
output_state <= 3'b000;
end
end
endcase



// Output spike transmission
if (output_spike_pending && m_axis_output_tready) begin
m_axis_output_tdata <= {8'h01, output_ch, output_y, output_x};
m_axis_output_tvalid <= 1'b1;
m_axis_output_tlast <= 1'b0;
output_spike_pending <= 1'b0;
end else if (!output_spike_pending) begin
m_axis_output_tvalid <= 1'b0;
end



// Apply decay to all accumulators (membrane leak)
if (time_window_counter[3:0] == 4'b0000) begin // Every 16 cycles
for (ch = 0; ch < INPUT_CHANNELS; ch = ch + 1) begin
for (y = 0; y < OUTPUT_HEIGHT; y = y + 1) begin
for (x = 0; x < OUTPUT_WIDTH; x = x + 1) begin
pool_accumulator[ch][y][x] <=
(pool_accumulator[ch][y][x] * decay_factor) >> 8;
end
end
end
end
end
end



// Output assignments
always @(*) begin
input_spike_count = local_input_count;
output_spike_count = local_output_count;
computation_done = (output_state == 3'b000) && !output_spike_pending;
end



endmodule
```

4. 1D Convolution Layer

Implements a 1D convolutional layer operating directly on binary spike trains, with configurable kernel size, stride, and padding.

Module: snn_conv1d

1D spiking convolution engine with configurable kernel/stride/padding.

```
module snn_conv1d #(
parameter INPUT_LENGTH = 128,
parameter INPUT_CHANNELS = 16,
parameter OUTPUT_CHANNELS = 32,
parameter KERNEL_SIZE = 3,
parameter STRIDE = 1,
parameter PADDING = 1,



// Calculated output length
parameter OUTPUT_LENGTH = (INPUT_LENGTH + 2*PADDING - KERNEL_SIZE) /
STRIDE + 1,



// Precision parameters
parameter WEIGHT_WIDTH = 8, // INT8 weights
parameter VMEM_WIDTH = 16, // Q8.8 membrane potential
parameter THRESHOLD = 16'h0100, // 1.0 in Q8.8
parameter LEAK_SHIFT = 4, // Leak = vmem >> 4 (≈0.9375 decay)



// Address widths (Verilog-2001 compatible)
parameter CH_ADDR_WIDTH = 5, // log2(32) = 5
parameter POS_ADDR_WIDTH = 7 // log2(128) = 7
)(
input wire clk,
input wire rst_n,
input wire enable,
//============================================================================
// Sparse Spike Input Interface (AXI-Stream)
// Only non-zero spikes are transmitted - zero spikes don't exist!
//============================================================================
input wire s_axis_spike_tvalid,
input wire [31:0] s_axis_spike_tdata, // {ch[15:8], pos[7:0], timestamp[15:0]}
input wire s_axis_spike_tlast,
output reg s_axis_spike_tready,



//============================================================================
// Sparse Spike Output Interface (AXI-Stream)
// Only output neurons that spike - exploits output sparsity!
//============================================================================
output reg m_axis_spike_tvalid,
output reg [31:0] m_axis_spike_tdata,
output reg m_axis_spike_tlast,
input wire m_axis_spike_tready,



//============================================================================
// Weight Memory Interface
//============================================================================
output reg weight_rd_en,
output reg [15:0] weight_addr,
input wire signed [WEIGHT_WIDTH-1:0] weight_data,
input wire weight_valid,



//============================================================================
// Configuration
//============================================================================
input wire [VMEM_WIDTH-1:0] config_threshold,
input wire config_valid,



//============================================================================
// Energy Monitoring Statistics
//============================================================================
output reg [31:0] input_spike_count,
```

```
output reg [31:0] output_spike_count,
output reg [31:0] ac_operation_count,
output reg [31:0] memory_access_count,
output reg [31:0] cycle_count,
output wire busy
);



//============================================================================
// State Machine
//============================================================================
localparam IDLE = 4'd0;
localparam RECEIVE_SPIKE = 4'd1;
localparam CALC_POSITIONS = 4'd2;
localparam FETCH_WEIGHT = 4'd3;
localparam WAIT_WEIGHT = 4'd4;
localparam ACCUMULATE = 4'd5;
localparam NEXT_KERNEL = 4'd6;
localparam NEXT_CHANNEL = 4'd7;
localparam CHECK_SPIKE = 4'd8;
localparam OUTPUT_SPIKE = 4'd9;
localparam APPLY_LEAK = 4'd10;
localparam DONE = 4'd11;



reg [3:0] state;
assign busy = (state != IDLE);



//============================================================================
// Input Spike Parsing
//============================================================================
reg [CH_ADDR_WIDTH-1:0] input_ch;
reg [POS_ADDR_WIDTH-1:0] input_pos;
reg [15:0] input_timestamp;



//============================================================================
// Convolution Counters
//============================================================================
reg [CH_ADDR_WIDTH-1:0] out_ch_counter;
reg [2:0] kernel_counter;
reg [POS_ADDR_WIDTH-1:0] out_pos;
//============================================================================
// Membrane Potential Memory (Flattened for Verilog-2001)
// Index = out_ch * OUTPUT_LENGTH + out_pos
//============================================================================
reg signed [VMEM_WIDTH-1:0] membrane_mem
[0:OUTPUT_CHANNELS*OUTPUT_LENGTH-1];



// Helper function for memory indexing
function [15:0] mem_idx;
input [CH_ADDR_WIDTH-1:0] ch;
input [POS_ADDR_WIDTH-1:0] pos;
begin
mem_idx = ch * OUTPUT_LENGTH + pos;
end
endfunction



//============================================================================
// Weight Address Calculation
// weight_addr = out_ch * (INPUT_CHANNELS * KERNEL_SIZE) +
// input_ch * KERNEL_SIZE + kernel_pos
//============================================================================
function [15:0] calc_weight_addr;
input [CH_ADDR_WIDTH-1:0] o_ch;
input [CH_ADDR_WIDTH-1:0] i_ch;
input [2:0] k_pos;
begin
calc_weight_addr = o_ch * (INPUT_CHANNELS * KERNEL_SIZE) +
i_ch * KERNEL_SIZE + k_pos;
end
endfunction



//============================================================================
// Current Threshold (configurable or default)
```

```
//============================================================================
reg [VMEM_WIDTH-1:0] current_threshold;



always @(posedge clk or negedge rst_n) begin
if (!rst_n)
current_threshold <= THRESHOLD;
else if (config_valid)
current_threshold <= config_threshold;
end



//============================================================================
// Memory Initialization
//============================================================================
integer i;
initial begin
for (i = 0; i < OUTPUT_CHANNELS * OUTPUT_LENGTH; i = i + 1) begin
membrane_mem[i] = 0;
end
end



//============================================================================
// Main Processing State Machine
//============================================================================
reg signed [VMEM_WIDTH-1:0] current_vmem;
reg signed [VMEM_WIDTH:0] vmem_after_ac;
reg [15:0] current_mem_idx;
reg [POS_ADDR_WIDTH-1:0] spike_output_pos;
reg [CH_ADDR_WIDTH-1:0] spike_output_ch;



always @(posedge clk or negedge rst_n) begin
if (!rst_n) begin
state <= IDLE;
s_axis_spike_tready <= 1'b1;
m_axis_spike_tvalid <= 1'b0;
m_axis_spike_tdata <= 32'b0;
m_axis_spike_tlast <= 1'b0;



weight_rd_en <= 1'b0;
weight_addr <= 16'b0;



input_spike_count <= 32'b0;
output_spike_count <= 32'b0;
ac_operation_count <= 32'b0;
memory_access_count <= 32'b0;
cycle_count <= 32'b0;



out_ch_counter <= 0;
kernel_counter <= 0;
out_pos <= 0;



end else if (enable) begin
cycle_count <= cycle_count + 1;



// Default outputs
m_axis_spike_tvalid <= 1'b0;
weight_rd_en <= 1'b0;



case (state)
//------------------------------------------------------------
// IDLE: Wait for input spike (zero energy when no spikes)
//------------------------------------------------------------
IDLE: begin
s_axis_spike_tready <= 1'b1;
if (s_axis_spike_tvalid) begin
state <= RECEIVE_SPIKE;
end
```

```
end



//============================================================
// RECEIVE_SPIKE: Parse incoming spike data
//============================================================
RECEIVE_SPIKE: begin
s_axis_spike_tready <= 1'b0;
input_ch <= s_axis_spike_tdata[23:16];
input_pos <= s_axis_spike_tdata[7:0];
input_timestamp <= s_axis_spike_tdata[15:0];
input_spike_count <= input_spike_count + 1;



out_ch_counter <= 0;
state <= CALC_POSITIONS;
end
//============================================================
// CALC_POSITIONS: Calculate which output positions are affected
//============================================================
CALC_POSITIONS: begin
kernel_counter <= 0;
// Calculate output position from input position
// out_pos = (input_pos + PADDING - kernel_idx) / STRIDE
out_pos <= (input_pos + PADDING) / STRIDE;
state <= FETCH_WEIGHT;
end



//============================================================
// FETCH_WEIGHT: Request weight from memory
//============================================================
FETCH_WEIGHT: begin
weight_addr <= calc_weight_addr(out_ch_counter, input_ch, kernel_counter);
weight_rd_en <= 1'b1;
memory_access_count <= memory_access_count + 1;
state <= WAIT_WEIGHT;
end



//============================================================
// WAIT_WEIGHT: Wait for memory read
//============================================================
WAIT_WEIGHT: begin
weight_rd_en <= 1'b0;
if (weight_valid) begin
state <= ACCUMULATE;
end
end



//============================================================
// ACCUMULATE: THE AC OPERATION!
//
// This is where energy savings happen:
// - Traditional MAC: vmem += spike * weight (multiply + add)
// - Our AC: vmem += weight (add only!)
//
// Since spike=1 (we only process valid spikes),
// spike * weight = weight, so multiply is eliminated!
//============================================================
ACCUMULATE: begin
// Calculate output position for this kernel element
if (out_pos >= kernel_counter && out_pos - kernel_counter < OUTPUT_LENGTH) begin
current_mem_idx <= mem_idx(out_ch_counter, out_pos - kernel_counter);
current_vmem <= membrane_mem[mem_idx(out_ch_counter, out_pos - kernel_counter)];



// AC OPERATION: Simply add weight (no multiply!)
vmem_after_ac <= membrane_mem[mem_idx(out_ch_counter, out_pos - kernel_counter)]
+ {{(VMEM_WIDTH-WEIGHT_WIDTH){weight_data[WEIGHT_WIDTH-1]}}, weight_data};



// Update membrane with saturation
if (vmem_after_ac[VMEM_WIDTH] != vmem_after_ac[VMEM_WIDTH-1]) begin
// Overflow - saturate
membrane_mem[current_mem_idx] <= vmem_after_ac[VMEM_WIDTH] ?
{1'b1, {(VMEM_WIDTH-1){1'b0}}} : {1'b0, {(VMEM_WIDTH-1){1'b1}}};
end else begin
```

```
membrane_mem[current_mem_idx] <= vmem_after_ac[VMEM_WIDTH-1:0];
end



ac_operation_count <= ac_operation_count + 1;
end



state <= NEXT_KERNEL;
end



//============================================================
// NEXT_KERNEL: Move to next kernel position
//============================================================
NEXT_KERNEL: begin
if (kernel_counter < KERNEL_SIZE - 1) begin
kernel_counter <= kernel_counter + 1;
state <= FETCH_WEIGHT;
end else begin
state <= NEXT_CHANNEL;
end
end



//============================================================
// NEXT_CHANNEL: Move to next output channel
//============================================================
NEXT_CHANNEL: begin
if (out_ch_counter < OUTPUT_CHANNELS - 1) begin
out_ch_counter <= out_ch_counter + 1;
kernel_counter <= 0;
state <= FETCH_WEIGHT;
end else begin
// Done processing this input spike
state <= CHECK_SPIKE;
spike_output_ch <= 0;
spike_output_pos <= 0;
end
end



//============================================================
// CHECK_SPIKE: Check if any neuron should fire
//============================================================
CHECK_SPIKE: begin
if (membrane_mem[mem_idx(spike_output_ch, spike_output_pos)] >=
$signed(current_threshold)) begin
state <= OUTPUT_SPIKE;
end else begin
// Move to next position
if (spike_output_pos < OUTPUT_LENGTH - 1) begin
spike_output_pos <= spike_output_pos + 1;
end else if (spike_output_ch < OUTPUT_CHANNELS - 1) begin
spike_output_pos <= 0;
spike_output_ch <= spike_output_ch + 1;
end else begin
// Done checking all neurons
state <= IDLE;
s_axis_spike_tready <= 1'b1;
end
end
end



//============================================================
// OUTPUT_SPIKE: Generate output spike (sparse output!)
//============================================================
OUTPUT_SPIKE: begin
if (m_axis_spike_tready) begin
m_axis_spike_tvalid <= 1'b1;
m_axis_spike_tdata <= {8'b0, spike_output_ch, spike_output_pos, input_timestamp};
output_spike_count <= output_spike_count + 1;



// Reset membrane potential after spike
membrane_mem[mem_idx(spike_output_ch, spike_output_pos)] <= 0;
```

```
// Continue checking
if (spike_output_pos < OUTPUT_LENGTH - 1) begin
spike_output_pos <= spike_output_pos + 1;
end else if (spike_output_ch < OUTPUT_CHANNELS - 1) begin
spike_output_pos <= 0;
spike_output_ch <= spike_output_ch + 1;
end else begin
state <= IDLE;
s_axis_spike_tready <= 1'b1;
end
state <= CHECK_SPIKE;
end
end



default: state <= IDLE;
endcase
end
end



endmodule
```

5. 1D Max Pooling

Implements 1D max pooling over spiking feature streams for spatial downsampling with sparse, event-driven inputs.

Module: snn_maxpool1d

1D max-pooling datapath for spiking inputs.

```
module snn_maxpool1d #(
parameter INPUT_LENGTH = 128,
parameter INPUT_CHANNELS = 32,
parameter POOL_SIZE = 2,
parameter STRIDE = 2,
parameter OUTPUT_LENGTH = INPUT_LENGTH / STRIDE,



// Precision parameters
parameter VMEM_WIDTH = 16, // Q8.8 membrane potential
parameter ADDR_WIDTH = 16,
parameter TIMESTAMP_WIDTH = 16
)(
input wire clk,
input wire rst_n,
input wire enable,



// Input spike interface (AXI-Stream)
input wire s_axis_input_tvalid,
input wire [31:0] s_axis_input_tdata,
input wire s_axis_input_tlast,
output reg s_axis_input_tready,



// Output spike interface (AXI-Stream)
output reg m_axis_output_tvalid,
output reg [31:0] m_axis_output_tdata,
output reg m_axis_output_tlast,
input wire m_axis_output_tready,



// Configuration
input wire config_valid,
input wire [31:0] config_data,



// Status
output reg busy,
output reg layer_done
);



// Max pooling memory for Verilog-2001 compatibility
reg [TIMESTAMP_WIDTH-1:0] max_timestamp_mem
[0:INPUT_CHANNELS*OUTPUT_LENGTH-1];
reg max_spike_mem [0:INPUT_CHANNELS*OUTPUT_LENGTH-1];



// Input spike buffer with timestamps
reg input_spike_buffer [0:INPUT_LENGTH-1];
reg [TIMESTAMP_WIDTH-1:0] input_timestamp_buffer [0:INPUT_LENGTH-1];
reg [7:0] input_channel_id;
reg [15:0] input_position;
reg [15:0] input_timestamp;



// Internal signals
reg [2:0] pool_state;
reg [15:0] current_ch;
reg [15:0] current_pos;
reg [TIMESTAMP_WIDTH-1:0] max_time;
reg has_spike;
integer k; // Moved outside always block for Verilog-2001
```

```
// State machine parameters
localparam IDLE = 3'b000;
localparam PROCESS_SPIKES = 3'b001;
localparam POOLING = 3'b010;
localparam OUTPUT_SPIKES = 3'b011;
localparam DONE = 3'b100;



// Helper function to calculate flattened index
function [31:0] pool_index;
input [15:0] ch;
input [15:0] pos;
begin
pool_index = ch * OUTPUT_LENGTH + pos;
end
endfunction



// Input spike parsing
always @(*) begin
if (s_axis_input_tvalid && s_axis_input_tready) begin
input_channel_id = s_axis_input_tdata[15:8];
input_position = s_axis_input_tdata[31:16];
input_timestamp = s_axis_input_tdata[15:0]; // Use lower bits for timestamp
end else begin
input_channel_id = 8'b0;
input_position = 16'b0;
input_timestamp = 16'b0;
end
end



// Main state machine
always @(posedge clk or negedge rst_n) begin
if (!rst_n) begin
pool_state <= IDLE;
current_ch <= 16'b0;
current_pos <= 16'b0;
max_time <= 16'b0;
has_spike <= 1'b0;



s_axis_input_tready <= 1'b0;
m_axis_output_tvalid <= 1'b0;
m_axis_output_tdata <= 32'b0;
m_axis_output_tlast <= 1'b0;



busy <= 1'b0;
layer_done <= 1'b0;



end else if (enable) begin
case (pool_state)
IDLE: begin
if (config_valid) begin
pool_state <= PROCESS_SPIKES;
s_axis_input_tready <= 1'b1;
busy <= 1'b1;
end else begin
s_axis_input_tready <= 1'b1;
busy <= 1'b0;
end
end



PROCESS_SPIKES: begin
if (s_axis_input_tvalid && s_axis_input_tready) begin
// Store input spike with timestamp
if (input_position < INPUT_LENGTH) begin
input_spike_buffer[input_position] <= 1'b1;
input_timestamp_buffer[input_position] <= input_timestamp;
end



if (s_axis_input_tlast) begin
pool_state <= POOLING;
s_axis_input_tready <= 1'b0;
```

```
current_ch <= 16'b0;
current_pos <= 16'b0;
end
end
end



POOLING: begin
// Perform 1D max pooling (find earliest/latest spike)
max_time <= 16'b0;
has_spike <= 1'b0;



// Find max (latest) spike in pool window
for (k = 0; k < POOL_SIZE; k = k + 1) begin
if ((current_pos * STRIDE + k) < INPUT_LENGTH) begin
if (input_spike_buffer[current_pos * STRIDE + k]) begin
if (!has_spike || input_timestamp_buffer[current_pos * STRIDE + k] > max_time) begin
max_time <= input_timestamp_buffer[current_pos * STRIDE + k];
has_spike <= 1'b1;
end
end
end
end



// Store max result
max_timestamp_mem[pool_index(current_ch, current_pos)] <= max_time;
max_spike_mem[pool_index(current_ch, current_pos)] <= has_spike;



// Move to next position
if (current_pos < OUTPUT_LENGTH - 1) begin
current_pos <= current_pos + 1;
end else begin
current_pos <= 16'b0;
if (current_ch < INPUT_CHANNELS - 1) begin
current_ch <= current_ch + 1;
end else begin
pool_state <= OUTPUT_SPIKES;
current_ch <= 16'b0;
current_pos <= 16'b0;
end
end
end



OUTPUT_SPIKES: begin
if (m_axis_output_tready) begin
// Check if we should output a spike
if (max_spike_mem[pool_index(current_ch, current_pos)]) begin
m_axis_output_tvalid <= 1'b1;
m_axis_output_tdata <= {current_pos, current_ch[7:0],
max_timestamp_mem[pool_index(current_ch, current_pos)][7:0], 1'b1};
end else begin
m_axis_output_tvalid <= 1'b0;
end



// Move to next position
if (current_pos < OUTPUT_LENGTH - 1) begin
current_pos <= current_pos + 1;
end else begin
current_pos <= 16'b0;
if (current_ch < INPUT_CHANNELS - 1) begin
current_ch <= current_ch + 1;
end else begin
m_axis_output_tlast <= 1'b1;
pool_state <= DONE;
end
end
end
end



DONE: begin
m_axis_output_tvalid <= 1'b0;
m_axis_output_tlast <= 1'b0;
busy <= 1'b0;
```

```
layer_done <= 1'b1;
pool_state <= IDLE;
end



default: begin
pool_state <= IDLE;
end
endcase
end
end



// Initialize memories
integer i;
initial begin
for (i = 0; i < INPUT_CHANNELS * OUTPUT_LENGTH; i = i + 1) begin
max_timestamp_mem[i] = 16'b0;
max_spike_mem[i] = 1'b0;
end



for (i = 0; i < INPUT_LENGTH; i = i + 1) begin
input_spike_buffer[i] = 1'b0;
input_timestamp_buffer[i] = 16'b0;
end
end



endmodule
```

6. k-Winners-Take-All (Lateral Inhibition)

Implements k-Winners-Take-All (k-WTA) lateral inhibition, restricting the number of simultaneously active neurons within a configurable inhibition radius — a key mechanism for unsupervised competitive learning in SNNs.

Module: k_winners_wta

k-Winners-Take-All lateral inhibition core.

```
module k_winners_wta #(
parameter HEIGHT = 10, // Feature map height
parameter WIDTH = 10, // Feature map width
parameter CHANNELS = 100, // Number of features/filters
parameter VMEM_WIDTH = 16, // Membrane potential precision
parameter MAX_WINNERS = 5, // Maximum k value
parameter MAX_RADIUS = 4 // Maximum inhibition radius
)(
input wire clk,
input wire rst_n,
input wire enable,



//============================================================================
// Control Interface
//============================================================================
input wire start,
input wire [$clog2(MAX_WINNERS):0] k, // Number of winners
input wire [$clog2(MAX_RADIUS):0] radius, // Inhibition radius
input wire [VMEM_WIDTH-1:0] threshold, // Minimum threshold
output reg done,
output wire busy,



//============================================================================
// Membrane Potential Input
// Assumes potentials are provided sequentially or via memory interface
//============================================================================
output reg vmem_rd_en,
output reg [$clog2(CHANNELS)-1:0] vmem_ch_addr,
output reg [$clog2(HEIGHT)-1:0] vmem_row_addr,
output reg [$clog2(WIDTH)-1:0] vmem_col_addr,
input wire [VMEM_WIDTH-1:0] vmem_data,
input wire vmem_valid,



//============================================================================
// Winner Output
//============================================================================
output reg winner_valid,
output reg [$clog2(CHANNELS)-1:0] winner_ch,
output reg [$clog2(HEIGHT)-1:0] winner_row,
output reg [$clog2(WIDTH)-1:0] winner_col,
output reg [VMEM_WIDTH-1:0] winner_potential,
output reg [$clog2(MAX_WINNERS):0] winner_index, // 0 to k-1
input wire winner_ready,



//============================================================================
// Spike Output (for all winners combined)
//============================================================================
output reg [CHANNELS*HEIGHT*WIDTH-1:0] spike_map, // Flattened spike output



//============================================================================
// Statistics
//============================================================================
output reg [$clog2(MAX_WINNERS):0] num_winners_found
);



//============================================================================
// Local Parameters
//============================================================================
localparam TOTAL_NEURONS = CHANNELS * HEIGHT * WIDTH;
localparam ADDR_WIDTH = $clog2(TOTAL_NEURONS);
```

```
//============================================================================
// State Machine
//============================================================================
localparam IDLE = 4'd0;
localparam INIT_SCAN = 4'd1;
localparam SCAN_READ = 4'd2;
localparam SCAN_WAIT = 4'd3;
localparam SCAN_COMPARE = 4'd4;
localparam SCAN_NEXT = 4'd5;
localparam CHECK_WINNER = 4'd6;
localparam APPLY_INHIBITION = 4'd7;
localparam OUTPUT_WINNER = 4'd8;
localparam NEXT_WINNER = 4'd9;
localparam FINISH = 4'd10;



reg [3:0] state;
assign busy = (state != IDLE);



//============================================================================
// Inhibition Mask Memory
// 1 = inhibited, 0 = can be selected
//============================================================================
reg [TOTAL_NEURONS-1:0] inhibition_mask;



//============================================================================
// Scan Registers
//============================================================================
reg [$clog2(CHANNELS)-1:0] scan_ch;
reg [$clog2(HEIGHT)-1:0] scan_row;
reg [$clog2(WIDTH)-1:0] scan_col;
reg scan_done;



// Current maximum tracking
reg [VMEM_WIDTH-1:0] current_max_potential;
reg [$clog2(CHANNELS)-1:0] current_max_ch;
reg [$clog2(HEIGHT)-1:0] current_max_row;
reg [$clog2(WIDTH)-1:0] current_max_col;
reg found_valid_winner;



// Winner count
reg [$clog2(MAX_WINNERS):0] winner_count;



// Inhibition application
reg signed [$clog2(HEIGHT)+1:0] inhib_row;
reg signed [$clog2(WIDTH)+1:0] inhib_col;



//============================================================================
// Address Calculation
//============================================================================
function [ADDR_WIDTH-1:0] flatten_addr;
input [$clog2(CHANNELS)-1:0] ch;
input [$clog2(HEIGHT)-1:0] row;
input [$clog2(WIDTH)-1:0] col;
begin
flatten_addr = ch * (HEIGHT * WIDTH) + row * WIDTH + col;
end
endfunction



//============================================================================
// Check if current scan position is inhibited
//============================================================================
wire current_inhibited;
wire [ADDR_WIDTH-1:0] current_addr = flatten_addr(scan_ch, scan_row, scan_col);
assign current_inhibited = inhibition_mask[current_addr];



//============================================================================
// Main State Machine
```

```
//=============================================================================
always @(posedge clk) begin
if (!rst_n) begin
state <= IDLE;
done <= 1'b0;
winner_valid <= 1'b0;
vmem_rd_en <= 1'b0;
inhibition_mask <= {TOTAL_NEURONS{1'b0}};
spike_map <= {TOTAL_NEURONS{1'b0}};
num_winners_found <= 0;
winner_count <= 0;
end else if (enable) begin
case (state)
//=============================================================
// IDLE: Wait for start signal
//=============================================================
IDLE: begin
done <= 1'b0;
winner_valid <= 1'b0;



if (start) begin
inhibition_mask <= {TOTAL_NEURONS{1'b0}};
spike_map <= {TOTAL_NEURONS{1'b0}};
winner_count <= 0;
num_winners_found <= 0;
state <= INIT_SCAN;
end
end



//=============================================================
// INIT_SCAN: Initialize scan for next winner
//=============================================================
INIT_SCAN: begin
scan_ch <= 0;
scan_row <= 0;
scan_col <= 0;
scan_done <= 1'b0;
current_max_potential <= 0;
found_valid_winner <= 1'b0;
state <= SCAN_READ;
end



//=============================================================
// SCAN_READ: Request membrane potential
//=============================================================
SCAN_READ: begin
if (!current_inhibited) begin
vmem_rd_en <= 1'b1;
vmem_ch_addr <= scan_ch;
vmem_row_addr <= scan_row;
vmem_col_addr <= scan_col;
state <= SCAN_WAIT;
end else begin
// Skip inhibited neuron
state <= SCAN_NEXT;
end
end



//=============================================================
// SCAN_WAIT: Wait for memory response
//=============================================================
SCAN_WAIT: begin
vmem_rd_en <= 1'b0;
if (vmem_valid) begin
state <= SCAN_COMPARE;
end
end



//=============================================================
// SCAN_COMPARE: Compare with current maximum
//=============================================================
SCAN_COMPARE: begin
// Check if this neuron is above threshold and is new max
if (vmem_data >= threshold && vmem_data > current_max_potential) begin
current_max_potential <= vmem_data;
current_max_ch <= scan_ch;
```

```
current_max_row <= scan_row;
current_max_col <= scan_col;
found_valid_winner <= 1'b1;
end
state <= SCAN_NEXT;
end



//============================================================
// SCAN_NEXT: Move to next neuron
//============================================================
SCAN_NEXT: begin
if (scan_col < WIDTH - 1) begin
scan_col <= scan_col + 1;
state <= SCAN_READ;
end else if (scan_row < HEIGHT - 1) begin
scan_col <= 0;
scan_row <= scan_row + 1;
state <= SCAN_READ;
end else if (scan_ch < CHANNELS - 1) begin
scan_col <= 0;
scan_row <= 0;
scan_ch <= scan_ch + 1;
state <= SCAN_READ;
end else begin
// Scan complete
state <= CHECK_WINNER;
end
end



//============================================================
// CHECK_WINNER: Check if valid winner found
//============================================================
CHECK_WINNER: begin
if (found_valid_winner) begin
// Found a winner
inhib_row <= $signed({1'b0, current_max_row}) - $signed({1'b0,
radius[$clog2(MAX_RADIUS):0]});
inhib_col <= $signed({1'b0, current_max_col}) - $signed({1'b0,
radius[$clog2(MAX_RADIUS):0]});
state <= APPLY_INHIBITION;
end else begin
// No more winners
state <= FINISH;
end
end



//============================================================
// APPLY_INHIBITION: Mark neighborhood as inhibited
//============================================================
APPLY_INHIBITION: begin
// Apply inhibition to all channels at spatial neighborhood
// This is done over multiple cycles for area efficiency



// For simplicity, do all at once (can be pipelined)
begin : inhibit_block
integer ch_i;
integer r, c;
reg signed [$clog2(HEIGHT)+1:0] row_i;
reg signed [$clog2(WIDTH)+1:0] col_i;
for (ch_i = 0; ch_i < CHANNELS; ch_i = ch_i + 1) begin
for (r = 0; r <= 2*MAX_RADIUS; r = r + 1) begin
for (c = 0; c <= 2*MAX_RADIUS; c = c + 1) begin
row_i = $signed({1'b0, current_max_row}) - $signed(radius) + r;
col_i = $signed({1'b0, current_max_col}) - $signed(radius) + c;



// Check bounds and within radius
if (row_i >= 0 && row_i < HEIGHT &&
col_i >= 0 && col_i < WIDTH &&
r <= 2*radius && c <= 2*radius) begin
inhibition_mask[ch_i * (HEIGHT * WIDTH) + row_i * WIDTH + col_i] <= 1'b1;
end
end
end
end
end
```

```
// Mark winner in spike map
spike_map[flatten_addr(current_max_ch, current_max_row, current_max_col)] <= 1'b1;


state <= OUTPUT_WINNER;
end


//============================================================
// OUTPUT_WINNER: Output winner information
//============================================================
OUTPUT_WINNER: begin
winner_valid <= 1'b1;
winner_ch <= current_max_ch;
winner_row <= current_max_row;
winner_col <= current_max_col;
winner_potential <= current_max_potential;
winner_index <= winner_count;


if (winner_ready) begin
winner_valid <= 1'b0;
winner_count <= winner_count + 1;
num_winners_found <= winner_count + 1;
state <= NEXT_WINNER;
end
end


//============================================================
// NEXT_WINNER: Check if more winners needed
//============================================================
NEXT_WINNER: begin
if (winner_count < k) begin
// Find next winner
state <= INIT_SCAN;
end else begin
// Found all k winners
state <= FINISH;
end
end


//============================================================
// FINISH: Signal completion
//============================================================
FINISH: begin
done <= 1'b1;
state <= IDLE;
end


default: state <= IDLE;
endcase
end
end


endmodule
```

7. Mozafari S1 Pipeline (DoG Temporal Encoding)

Implements the S1 (Difference-of-Gaussians edge-extraction) stage of the Mozafari et al. deep SNN model, converting static images into temporally coded spike trains, along with an AXI4-Lite wrapper for processor-system integration.

Module: mozafari_s1_pipeline

DoG-based S1 encoding pipeline (Mozafari model) producing temporally coded spikes.

```
module mozafari_s1_pipeline #(
parameter INPUT_HEIGHT = 28,
parameter INPUT_WIDTH = 28,
parameter NUM_DOG_FILTERS = 6,
parameter DATA_WIDTH = 8, // Input pixel precision
parameter DOG_OUT_WIDTH = 16, // DoG output precision
parameter TIME_STEPS = 15, // Temporal resolution
parameter TIME_WIDTH = 4, // log2(TIME_STEPS)
parameter DOG_THRESHOLD = 50, // DoG filter threshold
parameter LATENCY_THRESHOLD = 10 // Minimum for spike generation
)(
input wire clk,
input wire rst_n,
input wire enable,



//============================================================================
// Control Interface
//============================================================================
input wire start,
output wire done,
output wire busy,



//============================================================================
// Image Input (AXI-Stream)
//============================================================================
input wire s_axis_tvalid,
input wire [DATA_WIDTH-1:0] s_axis_tdata,
input wire s_axis_tlast,
output wire s_axis_tready,



//============================================================================
// Spike Output (AXI-Stream with temporal info)
//============================================================================
output wire m_axis_tvalid,
output wire [TIME_WIDTH-1:0] m_axis_tdata_time,
output wire [$clog2(NUM_DOG_FILTERS)-1:0] m_axis_tdata_ch,
output wire [$clog2(INPUT_HEIGHT)-1:0] m_axis_tdata_row,
output wire [$clog2(INPUT_WIDTH)-1:0] m_axis_tdata_col,
output wire m_axis_tdata_valid_spike, // Actually spiked
output wire m_axis_tlast,
input wire m_axis_tready,



//============================================================================
// Spike Memory Interface (for bulk read by downstream layers)
//============================================================================
output wire spike_mem_wen,
output wire
[$clog2(TIME_STEPS*INPUT_HEIGHT*INPUT_WIDTH*NUM_DOG_FILTERS)-1:0]
spike_mem_addr,
output wire spike_mem_data,



//============================================================================
// Configuration
//============================================================================
input wire [DOG_OUT_WIDTH-1:0] cfg_max_intensity,



//============================================================================
// Debug/Status
//============================================================================
output wire [31:0] stat_pixels_in,
output wire [31:0] stat_spikes_out,
```

```
output wire [31:0] stat_filtered_out,
output wire [1:0] pipeline_stage // 0=idle, 1=dog, 2=encode, 3=inhibit
);



//================================================================================
// Internal Signals
//================================================================================



// DoG Filter Bank outputs
wire dog_done;
wire dog_busy;
wire dog_pixel_valid;
wire [DOG_OUT_WIDTH-1:0] dog_pixel_data;
wire [$clog2(NUM_DOG_FILTERS)-1:0] dog_pixel_ch;
wire dog_pixel_last;
wire dog_frame_last;
wire dog_pixel_ready;
wire [31:0] dog_nonzero;



// Intensity-to-Latency outputs
wire latency_done;
wire latency_busy;
wire latency_spike_valid;
wire [TIME_WIDTH-1:0] latency_spike_time;
wire [$clog2(NUM_DOG_FILTERS)-1:0] latency_spike_ch;
wire [$clog2(INPUT_HEIGHT)-1:0] latency_spike_row;
wire [$clog2(INPUT_WIDTH)-1:0] latency_spike_col;
wire latency_no_spike;
wire latency_frame_last;
wire latency_spike_ready;
wire [31:0] latency_total_spikes;



// Pointwise Inhibition outputs
wire inhibit_done;
wire inhibit_busy;
wire inhibit_spike_valid;
wire [TIME_WIDTH-1:0] inhibit_spike_time;
wire [$clog2(NUM_DOG_FILTERS)-1:0] inhibit_spike_ch;
wire [$clog2(INPUT_HEIGHT)-1:0] inhibit_spike_row;
wire [$clog2(INPUT_WIDTH)-1:0] inhibit_spike_col;
wire inhibit_was_first;
wire inhibit_frame_last;



// Spike memory write signals
wire inhibit_mem_write_en;
wire [$clog2(TIME_STEPS)-1:0] inhibit_mem_time;
wire [$clog2(INPUT_HEIGHT)-1:0] inhibit_mem_row;
wire [$clog2(INPUT_WIDTH)-1:0] inhibit_mem_col;
wire [$clog2(NUM_DOG_FILTERS)-1:0] inhibit_mem_ch;



//================================================================================
// Pipeline Control
//================================================================================
reg [1:0] current_stage;
reg pipeline_done;
assign done = pipeline_done;
assign busy = dog_busy | latency_busy | inhibit_busy;
assign pipeline_stage = current_stage;



// Statistics
wire [31:0] dog_pixels_processed;
assign stat_pixels_in = dog_pixels_processed;
assign stat_filtered_out = dog_nonzero;
assign stat_spikes_out = latency_total_spikes;



//================================================================================
// Stage 1: DoG Filter Bank
//================================================================================
dog_filter_bank #(
.INPUT_HEIGHT(INPUT_HEIGHT),
```

```
.INPUT_WIDTH(INPUT_WIDTH),
.NUM_FILTERS(NUM_DOG_FILTERS),
.DATA_WIDTH(DATA_WIDTH),
.OUTPUT_WIDTH(DOG_OUT_WIDTH),
.THRESHOLD(DOG_THRESHOLD)
) u_dog_filter (
.clk(clk),
.rst_n(rst_n),
.enable(enable),
.start(start),
.done(dog_done),
.busy(dog_busy),
// Input
.s_pixel_valid(s_axis_tvalid),
.s_pixel_data(s_axis_tdata),
.s_pixel_last(s_axis_tlast),
.s_frame_last(1'b0),
.s_pixel_ready(s_axis_tready),
// Output
.m_pixel_valid(dog_pixel_valid),
.m_pixel_data(dog_pixel_data),
.m_pixel_ch(dog_pixel_ch),
.m_pixel_last(dog_pixel_last),
.m_frame_last(dog_frame_last),
.m_pixel_ready(dog_pixel_ready),
// Stats
.pixels_processed(dog_pixels_processed),
.nonzero_outputs(dog_nonzero)
);



//============================================================================
// Stage 2: Intensity-to-Latency Encoder
//============================================================================
intensity_to_latency #(
.HEIGHT(INPUT_HEIGHT),
.WIDTH(INPUT_WIDTH),
.CHANNELS(NUM_DOG_FILTERS),
.DATA_WIDTH(DOG_OUT_WIDTH),
.TIME_STEPS(TIME_STEPS),
.TIME_WIDTH(TIME_WIDTH),
.THRESHOLD(LATENCY_THRESHOLD),
.MODE("LINEAR")
) u_latency_encoder (
.clk(clk),
.rst_n(rst_n),
.enable(enable),
.start(dog_done), // Chain to DoG completion
.done(latency_done),
.busy(latency_busy),
// Input from DoG
.s_intensity_valid(dog_pixel_valid),
.s_intensity_data(dog_pixel_data),
.s_intensity_ch(dog_pixel_ch),
.s_frame_last(dog_frame_last),
.s_intensity_ready(dog_pixel_ready),
// Output
.m_spike_valid(latency_spike_valid),
.m_spike_time(latency_spike_time),
.m_spike_ch(latency_spike_ch),
.m_spike_row(latency_spike_row),
.m_spike_col(latency_spike_col),
.m_no_spike(latency_no_spike),
.m_frame_last(latency_frame_last),
.m_spike_ready(latency_spike_ready),
// Config
.max_intensity(cfg_max_intensity),
.alpha(8'd16),
// Stats
.total_spikes(latency_total_spikes)
);



//============================================================================
// Stage 3: Pointwise Inhibition
//============================================================================
pointwise_inhibition #(
.HEIGHT(INPUT_HEIGHT),
.WIDTH(INPUT_WIDTH),
.CHANNELS(NUM_DOG_FILTERS),
.TIME_STEPS(TIME_STEPS),
.TIME_WIDTH(TIME_WIDTH)
) u_pointwise_inhibit (
```

```
.clk(clk),
.rst_n(rst_n),
.enable(enable),
.start(latency_done), // Chain to latency encoder completion
.done(inhibit_done),
.busy(inhibit_busy),
// Input from Latency Encoder
.s_spike_valid(latency_spike_valid & ~latency_no_spike),
.s_spike_time(latency_spike_time),
.s_spike_ch(latency_spike_ch),
.s_spike_row(latency_spike_row),
.s_spike_col(latency_spike_col),
.s_frame_last(latency_frame_last),
.s_spike_ready(latency_spike_ready),
// Output
.m_spike_valid(inhibit_spike_valid),
.m_spike_time(inhibit_spike_time),
.m_spike_ch(inhibit_spike_ch),
.m_spike_row(inhibit_spike_row),
.m_spike_col(inhibit_spike_col),
.m_was_first(inhibit_was_first),
.m_frame_last(inhibit_frame_last),
.m_spike_ready(m_axis_tready)
);



//============================================================================
// Output Assignment
//============================================================================
assign m_axis_tvalid = inhibit_spike_valid;
assign m_axis_tdata_time = inhibit_spike_time;
assign m_axis_tdata_ch = inhibit_spike_ch;
assign m_axis_tdata_row = inhibit_spike_row;
assign m_axis_tdata_col = inhibit_spike_col;
assign m_axis_tdata_valid_spike = inhibit_was_first; // Only first spikes pass
assign m_axis_tlast = inhibit_frame_last;



//============================================================================
// Spike Memory Write
// Address = time * (H*W*C) + row * (W*C) + col * C + ch
//============================================================================
assign spike_mem_wen = inhibit_spike_valid & inhibit_was_first & m_axis_tready;
assign spike_mem_addr = inhibit_spike_time * (INPUT_HEIGHT * INPUT_WIDTH *
NUM_DOG_FILTERS) +
inhibit_spike_row * (INPUT_WIDTH * NUM_DOG_FILTERS) +
inhibit_spike_col * NUM_DOG_FILTERS +
inhibit_spike_ch;
assign spike_mem_data = 1'b1;



//============================================================================
// Pipeline Stage Tracking
//============================================================================
always @(posedge clk) begin
if (!rst_n) begin
current_stage <= 2'd0;
pipeline_done <= 1'b0;
end else if (enable) begin
pipeline_done <= 1'b0;



if (start) begin
current_stage <= 2'd1;
end else if (dog_done && !latency_busy) begin
current_stage <= 2'd2;
end else if (latency_done && !inhibit_busy) begin
current_stage <= 2'd3;
end else if (inhibit_done) begin
current_stage <= 2'd0;
pipeline_done <= 1'b1;
end
end
end



endmodule



//----------------------------------------------------------------------------------------------------------------------
```

```
// Top-level Integration with AXI4-Lite Control
//------------------------------------------------------------------------------------------------------------------------
```

Module: mozafari_s1_axi_wrapper

AXI4-Lite peripheral wrapper for the Mozafari S1 pipeline.

```
module mozafari_s1_axi_wrapper #(
parameter INPUT_HEIGHT = 28,
parameter INPUT_WIDTH = 28,
parameter NUM_DOG_FILTERS = 6,
parameter TIME_STEPS = 15,
parameter C_S_AXI_DATA_WIDTH = 32,
parameter C_S_AXI_ADDR_WIDTH = 6
)(
input wire aclk,
input wire aresetn,



//============================================================================
// AXI4-Lite Control Interface
//============================================================================
input wire [C_S_AXI_ADDR_WIDTH-1:0] s_axi_awaddr,
input wire s_axi_awvalid,
output wire s_axi_awready,
input wire [C_S_AXI_DATA_WIDTH-1:0] s_axi_wdata,
input wire [C_S_AXI_DATA_WIDTH/8-1:0] s_axi_wstrb,
input wire s_axi_wvalid,
output wire s_axi_wready,
output wire [1:0] s_axi_bresp,
output wire s_axi_bvalid,
input wire s_axi_bready,
input wire [C_S_AXI_ADDR_WIDTH-1:0] s_axi_araddr,
input wire s_axi_arvalid,
output wire s_axi_arready,
output wire [C_S_AXI_DATA_WIDTH-1:0] s_axi_rdata,
output wire [1:0] s_axi_rresp,
output wire s_axi_rvalid,
input wire s_axi_rready,



//============================================================================
// AXI-Stream Input (Image pixels)
//============================================================================
input wire [7:0] s_axis_tdata,
input wire s_axis_tvalid,
input wire s_axis_tlast,
output wire s_axis_tready,



//============================================================================
// AXI-Stream Output (Temporal spikes)
//============================================================================
output wire [31:0] m_axis_tdata, // Packed spike info
output wire m_axis_tvalid,
output wire m_axis_tlast,
input wire m_axis_tready,



//============================================================================
// Interrupt
//============================================================================
output wire irq_done
);



//============================================================================
// Register Map
//============================================================================
// 0x00: Control (W) - bit0: start, bit1: enable, bit2: reset
// 0x04: Status (R) - bit0: done, bit1: busy, bit[3:2]: stage
// 0x08: Config (RW) - max_intensity[15:0], thresholds
// 0x0C: Stat0 (R) - pixels_in
// 0x10: Stat1 (R) - spikes_out
// 0x14: Stat2 (R) - filtered_out



reg [C_S_AXI_DATA_WIDTH-1:0] reg_control;
```

```
reg [C_S_AXI_DATA_WIDTH-1:0] reg_config;
wire [C_S_AXI_DATA_WIDTH-1:0] reg_status;
wire [31:0] stat_pixels, stat_spikes, stat_filtered;
wire [1:0] stage;



wire core_start;
wire core_enable;
wire core_done;
wire core_busy;



assign core_start = reg_control[0];
assign core_enable = reg_control[1];
assign reg_status = {28'b0, stage, core_busy, core_done};
assign irq_done = core_done;



// Core outputs
wire [3:0] spike_time;
wire [2:0] spike_ch;
wire [4:0] spike_row, spike_col;
wire spike_valid_flag;



//============================================================================
// S1 Pipeline Core
//============================================================================
mozafari_s1_pipeline #(
.INPUT_HEIGHT(INPUT_HEIGHT),
.INPUT_WIDTH(INPUT_WIDTH),
.NUM_DOG_FILTERS(NUM_DOG_FILTERS),
.TIME_STEPS(TIME_STEPS)
) u_s1_pipeline (
.clk(aclk),
.rst_n(aresetn),
.enable(core_enable),
.start(core_start),
.done(core_done),
.busy(core_busy),
// Image input
.s_axis_tvalid(s_axis_tvalid),
.s_axis_tdata(s_axis_tdata),
.s_axis_tlast(s_axis_tlast),
.s_axis_tready(s_axis_tready),
// Spike output
.m_axis_tvalid(m_axis_tvalid),
.m_axis_tdata_time(spike_time),
.m_axis_tdata_ch(spike_ch),
.m_axis_tdata_row(spike_row),
.m_axis_tdata_col(spike_col),
.m_axis_tdata_valid_spike(spike_valid_flag),
.m_axis_tlast(m_axis_tlast),
.m_axis_tready(m_axis_tready),
// Config
.cfg_max_intensity(reg_config[15:0]),
// Stats
.stat_pixels_in(stat_pixels),
.stat_spikes_out(stat_spikes),
.stat_filtered_out(stat_filtered),
.pipeline_stage(stage)
);



// Pack spike output
// [31:28] = time, [27:25] = ch, [24:20] = row, [19:15] = col, [0] = valid
assign m_axis_tdata = {spike_time, spike_ch, spike_row, spike_col, 14'b0,
spike_valid_flag};



//============================================================================
// AXI4-Lite Interface (Simplified)
//============================================================================
reg axi_awready, axi_wready, axi_bvalid;
reg axi_arready, axi_rvalid;
reg [C_S_AXI_DATA_WIDTH-1:0] axi_rdata;
reg [C_S_AXI_ADDR_WIDTH-1:0] axi_awaddr, axi_araddr;
```

```
assign s_axi_awready = axi_awready;
assign s_axi_wready = axi_wready;
assign s_axi_bresp = 2'b00;
assign s_axi_bvalid = axi_bvalid;
assign s_axi_arready = axi_arready;
assign s_axi_rdata = axi_rdata;
assign s_axi_rresp = 2'b00;
assign s_axi_rvalid = axi_rvalid;



// Write logic
always @(posedge aclk) begin
if (!aresetn) begin
axi_awready <= 1'b0;
axi_wready <= 1'b0;
axi_bvalid <= 1'b0;
reg_control <= 0;
reg_config <= 32'h00FF0032; // Default: max=255, threshold=50
end else begin
// Write address ready
if (~axi_awready && s_axi_awvalid && s_axi_wvalid) begin
axi_awready <= 1'b1;
axi_awaddr <= s_axi_awaddr;
end else begin
axi_awready <= 1'b0;
end



// Write data ready
if (~axi_wready && s_axi_awvalid && s_axi_wvalid) begin
axi_wready <= 1'b1;
end else begin
axi_wready <= 1'b0;
end
// Write response
if (axi_awready && s_axi_awvalid && axi_wready && s_axi_wvalid && ~axi_bvalid) begin
axi_bvalid <= 1'b1;



// Register write
case (axi_awaddr[5:2])
4'h0: reg_control <= s_axi_wdata;
4'h2: reg_config <= s_axi_wdata;
endcase
end else if (s_axi_bready && axi_bvalid) begin
axi_bvalid <= 1'b0;
end



// Auto-clear start bit
if (reg_control[0]) begin
reg_control[0] <= 1'b0;
end
end
end



// Read logic
always @(posedge aclk) begin
if (!aresetn) begin
axi_arready <= 1'b0;
axi_rvalid <= 1'b0;
axi_rdata <= 0;
end else begin
if (~axi_arready && s_axi_arvalid) begin
axi_arready <= 1'b1;
axi_araddr <= s_axi_araddr;
end else begin
axi_arready <= 1'b0;
end



if (axi_arready && s_axi_arvalid && ~axi_rvalid) begin
axi_rvalid <= 1'b1;



case (axi_araddr[5:2])
4'h0: axi_rdata <= reg_control;
4'h1: axi_rdata <= reg_status;
```

```
4'h2: axi_rdata <= reg_config;
4'h3: axi_rdata <= stat_pixels;
4'h4: axi_rdata <= stat_spikes;
4'h5: axi_rdata <= stat_filtered;
default: axi_rdata <= 0;
endcase
end else if (axi_rvalid && s_axi_rready) begin
axi_rvalid <= 1'b0;
end
end
end



endmodule
```

8. STDP Learning Engine

Implements Spike-Timing-Dependent Plasticity (STDP), an unsupervised, biologically inspired learning rule that updates synaptic weights based on the relative timing of pre- and post-synaptic spikes, plus the supporting weight-memory block.

Module: stdp_engine

STDP weight-update engine driven by pre/post spike timing.

```
module stdp_engine #(
parameter KERNEL_SIZE = 5,
parameter IN_CHANNELS = 6,
parameter OUT_FEATURES = 20,
parameter WEIGHT_WIDTH = 8, // Q0.8 fixed point (0-1 range)
parameter TIME_WIDTH = 4,
parameter NO_SPIKE_VAL = 4'hF, // Indicates no spike
parameter NUM_SYNAPSES = KERNEL_SIZE * KERNEL_SIZE * IN_CHANNELS
)(
input wire clk,
input wire rst_n,
input wire enable,



//============================================================================
// Control
//============================================================================
input wire start,
output reg done,
output wire busy,



//============================================================================
// Learning Parameters (from software, Q8.8 format)
//============================================================================
input wire [15:0] cfg_a_plus, // LTP rate
input wire [15:0] cfg_a_minus, // LTD rate
input wire [WEIGHT_WIDTH-1:0] cfg_w_min, // Min weight (usually 0)
input wire [WEIGHT_WIDTH-1:0] cfg_w_max, // Max weight (usually 255)
input wire [7:0] cfg_mu, // Exponent (Q4.4, usually 0.75)



//============================================================================
// Reward Signal (for R-STDP)
//============================================================================
input wire reward_valid,
input wire reward_positive, // 1=correct, 0=incorrect
input wire [7:0] reward_magnitude, // Reward strength
input wire use_anti_stdp, // Apply anti-STDP on wrong



//============================================================================
// Winner Information
//============================================================================
input wire winner_valid,
input wire [$clog2(OUT_FEATURES)-1:0] winner_id,
input wire [TIME_WIDTH-1:0] winner_spike_time,
input wire [$clog2(28)-1:0] winner_row, // Position in output
input wire [$clog2(28)-1:0] winner_col,



//============================================================================
// Pre-synaptic Spike Times (from pointwise_inhibition_v2)
// For the receptive field of the winner neuron
//============================================================================
input wire pre_times_valid,
input wire [TIME_WIDTH*NUM_SYNAPSES-1:0] pre_spike_times, // Packed array



//============================================================================
// Weight Memory Interface
//============================================================================
// Read weights for winner
output reg weight_rd_en,
output reg [$clog2(OUT_FEATURES)-1:0] weight_rd_feature,
output reg [$clog2(NUM_SYNAPSES)-1:0] weight_rd_synapse,
input wire [WEIGHT_WIDTH-1:0] weight_rd_data,
```

```
input wire weight_rd_valid,



// Write updated weights
output reg weight_wr_en,
output reg [$clog2(OUT_FEATURES)-1:0] weight_wr_feature,
output reg [$clog2(NUM_SYNAPSES)-1:0] weight_wr_synapse,
output reg [WEIGHT_WIDTH-1:0] weight_wr_data,



//============================================================================
// Statistics
//============================================================================
output reg [31:0] ltp_count,
output reg [31:0] ltd_count,
output reg [31:0] no_update_count,
output reg [31:0] anti_stdp_count
);



//============================================================================
// State Machine
//============================================================================
localparam IDLE = 4'd0;
localparam WAIT_PRE = 4'd1;
localparam READ_WEIGHT = 4'd2;
localparam WAIT_READ = 4'd3;
localparam COMPUTE = 4'd4;
localparam WRITE_WEIGHT = 4'd5;
localparam NEXT_SYNAPSE = 4'd6;
localparam DONE_STATE = 4'd7;



reg [3:0] state;
assign busy = (state != IDLE);



//============================================================================
// Working Registers
//============================================================================
reg [$clog2(OUT_FEATURES)-1:0] current_feature;
reg [$clog2(NUM_SYNAPSES)-1:0] current_synapse;
reg [TIME_WIDTH-1:0] post_time; // Winner spike time
reg [TIME_WIDTH-1:0] pre_time; // Current synapse spike time
reg [WEIGHT_WIDTH-1:0] current_weight;
reg is_reward;
reg do_anti_stdp;



// Unpack pre-synaptic times
wire [TIME_WIDTH-1:0] pre_times [0:NUM_SYNAPSES-1];
genvar g;
generate
for (g = 0; g < NUM_SYNAPSES; g = g + 1) begin : unpack_times
assign pre_times[g] = pre_spike_times[g*TIME_WIDTH +: TIME_WIDTH];
end
endgenerate



//============================================================================
// Weight Update Computation
// Supports μ parameter via shift-based power approximation:
// cfg_mu = Q4.4 format: 0x10 = 1.0 (linear), 0x0C = 0.75, 0x08 = 0.5
// For μ < 1.0: attenuate the weight-dependent factor
// μ=1.0: delta = a * distance / 256 (linear, original)
// μ=0.75: delta = a * distance * 3/4 / 256 (approximate)
// μ=0.5: delta = a * sqrt(distance) / 256 (approximate via shift)
//
// LTP: Δw = a_plus * (w_max - w)^μ >> 8
// LTD: Δw = a_minus * (w - w_min)^μ >> 8
//============================================================================
reg signed [WEIGHT_WIDTH+16:0] delta_w;
reg [WEIGHT_WIDTH-1:0] new_weight;
reg update_type; // 0=LTD, 1=LTP



// Power approximation: apply cfg_mu to distance value
// cfg_mu Q4.4: 0x10=1.0, 0x0C=0.75, 0x08=0.5, 0x04=0.25
```

```
function [WEIGHT_WIDTH-1:0] apply_mu;
input [WEIGHT_WIDTH-1:0] distance;
input [7:0] mu; // Q4.4
reg [WEIGHT_WIDTH+3:0] scaled;
begin
// mu_frac = mu[3:0] (fractional 4 bits), mu_int = mu[7:4]
// For mu_int=1: full distance; mu_int=0: fractional only
// Approximation: result = distance * mu / 16
scaled = distance * mu[7:0];
apply_mu = scaled[WEIGHT_WIDTH+3:4]; // divide by 16 (Q4.4 to integer)
end
endfunction



// Compute weight update (combinational)
always @(*) begin
if (is_reward || !use_anti_stdp) begin
// Normal STDP or positive reward
if (pre_time <= post_time && pre_time != NO_SPIKE_VAL) begin
// LTP: pre fired before or same time as post
update_type = 1'b1;
delta_w = (cfg_a_plus * apply_mu(cfg_w_max - current_weight, cfg_mu)) >>> 8;
end else if (pre_time > post_time && pre_time != NO_SPIKE_VAL) begin
// LTD: pre fired after post
update_type = 1'b0;
delta_w = (cfg_a_minus * apply_mu(current_weight - cfg_w_min, cfg_mu)) >>> 8;
end else begin
// No spike from pre - no update
update_type = 1'b0;
delta_w = 0;
end
end else begin
// Anti-STDP (incorrect response)
if (pre_time <= post_time && pre_time != NO_SPIKE_VAL) begin
// Would be LTP -> becomes LTD
update_type = 1'b0;
delta_w = (cfg_a_minus * apply_mu(current_weight - cfg_w_min, cfg_mu)) >>> 8;
end else if (pre_time > post_time && pre_time != NO_SPIKE_VAL) begin
// Would be LTD -> becomes LTP
update_type = 1'b1;
delta_w = (cfg_a_plus * apply_mu(cfg_w_max - current_weight, cfg_mu)) >>> 8;
end else begin
update_type = 1'b0;
delta_w = 0;
end
end
end



// Apply update with saturation
always @(*) begin
if (update_type) begin
// LTP - increase weight
if (current_weight + delta_w > cfg_w_max) begin
new_weight = cfg_w_max;
end else begin
new_weight = current_weight + delta_w[WEIGHT_WIDTH-1:0];
end
end else begin
// LTD - decrease weight
if (delta_w > current_weight - cfg_w_min) begin
new_weight = cfg_w_min;
end else begin
new_weight = current_weight - delta_w[WEIGHT_WIDTH-1:0];
end
end
end



//============================================================================
// Main State Machine
//============================================================================
always @(posedge clk) begin
if (!rst_n) begin
state <= IDLE;
done <= 1'b0;
weight_rd_en <= 1'b0;
weight_wr_en <= 1'b0;
ltp_count <= 0;
ltd_count <= 0;
no_update_count <= 0;
anti_stdp_count <= 0;
```

```
end else if (enable) begin
case (state)
//============================================================
// IDLE: Wait for start
//============================================================
IDLE: begin
done <= 1'b0;
weight_rd_en <= 1'b0;
weight_wr_en <= 1'b0;



if (start && winner_valid) begin
current_feature <= winner_id;
post_time <= winner_spike_time;
current_synapse <= 0;



// Latch reward info
is_reward <= reward_valid && reward_positive;
do_anti_stdp <= reward_valid && !reward_positive && use_anti_stdp;



state <= WAIT_PRE;
end
end



//============================================================
// WAIT_PRE: Wait for pre-synaptic spike times
//============================================================
WAIT_PRE: begin
if (pre_times_valid) begin
state <= READ_WEIGHT;
end
end



//============================================================
// READ_WEIGHT: Request weight read
//============================================================
READ_WEIGHT: begin
weight_rd_en <= 1'b1;
weight_rd_feature <= current_feature;
weight_rd_synapse <= current_synapse;



// Get pre spike time for current synapse
pre_time <= pre_times[current_synapse];



state <= WAIT_READ;
end



//============================================================
// WAIT_READ: Wait for weight data
//============================================================
WAIT_READ: begin
weight_rd_en <= 1'b0;



if (weight_rd_valid) begin
current_weight <= weight_rd_data;
state <= COMPUTE;
end
end



//============================================================
// COMPUTE: Compute weight update
//============================================================
COMPUTE: begin
// Update statistics
if (pre_time == NO_SPIKE_VAL || post_time == NO_SPIKE_VAL) begin
no_update_count <= no_update_count + 1;
end else if (do_anti_stdp) begin
```

```
anti_stdp_count <= anti_stdp_count + 1;
if (update_type) ltp_count <= ltp_count + 1;
else ltd_count <= ltd_count + 1;
end else begin
if (update_type) ltp_count <= ltp_count + 1;
else ltd_count <= ltd_count + 1;
end
state <= WRITE_WEIGHT;
end



//============================================================
// WRITE_WEIGHT: Write updated weight
//============================================================
WRITE_WEIGHT: begin
// Only write if there was an actual update
if (delta_w != 0) begin
weight_wr_en <= 1'b1;
weight_wr_feature <= current_feature;
weight_wr_synapse <= current_synapse;
weight_wr_data <= new_weight;
end



state <= NEXT_SYNAPSE;
end



//============================================================
// NEXT_SYNAPSE: Move to next synapse
//============================================================
NEXT_SYNAPSE: begin
weight_wr_en <= 1'b0;



if (current_synapse < NUM_SYNAPSES - 1) begin
current_synapse <= current_synapse + 1;
state <= READ_WEIGHT;
end else begin
state <= DONE_STATE;
end
end



//============================================================
// DONE_STATE: Complete
//============================================================
DONE_STATE: begin
done <= 1'b1;
weight_wr_en <= 1'b0;
state <= IDLE;
end



default: state <= IDLE;
endcase
end
end



endmodule



//----------------------------------------------------------------------------------------------------------------------
// Weight Memory Module (BRAM-based)
//----------------------------------------------------------------------------------------------------------------------
```

Module: stdp_weight_memory

Memory block storing STDP-trained synaptic weights.

```
module stdp_weight_memory #(
parameter NUM_FEATURES = 20,
parameter NUM_SYNAPSES = 150, // 5x5x6
parameter WEIGHT_WIDTH = 8
)(
```

```
input wire clk,
input wire rst_n,



//  Port  A:  Read
input wire rd_en,
input wire [$clog2(NUM_FEATURES)-1:0] rd_feature,
input wire [$clog2(NUM_SYNAPSES)-1:0] rd_synapse,
output reg [WEIGHT_WIDTH-1:0] rd_data,
output reg rd_valid,



// Port B: Write
input wire wr_en,
input wire [$clog2(NUM_FEATURES)-1:0] wr_feature,
input wire [$clog2(NUM_SYNAPSES)-1:0] wr_synapse,
input wire [WEIGHT_WIDTH-1:0] wr_data,



// Initialization interface
input wire init_en,
input wire [$clog2(NUM_FEATURES*NUM_SYNAPSES)-1:0] init_addr,
input wire [WEIGHT_WIDTH-1:0] init_data
);



localparam MEM_SIZE = NUM_FEATURES * NUM_SYNAPSES;
localparam ADDR_WIDTH = $clog2(MEM_SIZE);



// Weight memory
reg [WEIGHT_WIDTH-1:0] weights [0:MEM_SIZE-1];



// Address calculation
wire [ADDR_WIDTH-1:0] rd_addr = rd_feature * NUM_SYNAPSES + rd_synapse;
wire [ADDR_WIDTH-1:0] wr_addr = wr_feature * NUM_SYNAPSES + wr_synapse;



// Initialize to default values
integer i;
initial begin
for (i = 0; i < MEM_SIZE; i = i + 1) begin
weights[i] = 8'd128; // 0.5 in Q0.8
end
end



// Read port
always @(posedge clk) begin
if (!rst_n) begin
rd_valid <= 1'b0;
rd_data <= 0;
end else if (rd_en) begin
rd_data <= weights[rd_addr];
rd_valid <= 1'b1;
end else begin
rd_valid <= 1'b0;
end
end



// Write port
always @(posedge clk) begin
if (init_en) begin
weights[init_addr] <= init_data;
end else if (wr_en) begin
weights[wr_addr] <= wr_data;
end
end



endmodule
```

9. LIF Neuron Array

Instantiates a parallel array of LIF neurons with shared axon fan-in, forming the core computational fabric of the accelerator.

Module: lif_neuron_array

Parallel array of LIF neurons with shared axon fan-in.

```
module lif_neuron_array #(
parameter NUM_NEURONS = 256,
parameter NUM_AXONS = 256,
parameter DATA_WIDTH = 16,
parameter WEIGHT_WIDTH = 8,
parameter THRESHOLD_WIDTH = 16,
parameter LEAK_WIDTH = 8,
parameter REFRAC_WIDTH = 8,
parameter NUM_PARALLEL_UNITS = 4, // Reduced from 8 to save LUT
parameter SPIKE_BUFFER_DEPTH = 64,
parameter USE_BRAM = 1,
parameter USE_DSP = 1,
parameter NEURON_ID_WIDTH = $clog2(NUM_NEURONS)
)(
input wire clk,
input wire rst_n,
input wire enable,



// Input spike interface (AXI-Stream)
input wire s_axis_spike_valid,
input wire [NEURON_ID_WIDTH-1:0] s_axis_spike_dest_id,
input wire [WEIGHT_WIDTH-1:0] s_axis_spike_weight,
input wire s_axis_spike_exc_inh,
output wire s_axis_spike_ready,



// Output spike interface
output reg m_axis_spike_valid,
output reg [NEURON_ID_WIDTH-1:0] m_axis_spike_neuron_id,
input wire m_axis_spike_ready,



// Configuration interface
input wire config_we,
input wire [NEURON_ID_WIDTH-1:0] config_addr,
input wire [31:0] config_data,



// Global neuron parameters
input wire [THRESHOLD_WIDTH-1:0] global_threshold,
input wire [LEAK_WIDTH-1:0] global_leak_rate,
input wire [REFRAC_WIDTH-1:0] global_refrac_period,



// Status
output wire [31:0] spike_count,
output wire array_busy,
output wire [31:0] throughput_counter,
output wire [7:0] active_neurons
);



localparam STATE_WIDTH = DATA_WIDTH + REFRAC_WIDTH; // 24 bits



//========================================================================
// Neuron State: Simple Dual-Port BRAM (UG901 SDP template)
//========================================================================
// Port A (write-only): leak writes + spike writes + config writes
// Port B (read-only): leak reads + spike reads
// Format: [23:8] = membrane (16-bit), [7:0] = refractory (8-bit)
// SDP guarantees BRAM inference at any depth (1024+) unlike TDP
//========================================================================
// SDP guarantees BRAM inference at any depth (1024+) when active
// Vivado auto-selects: BRAM when memory has real data flow, optimized away when idle
reg [STATE_WIDTH-1:0] neuron_state_mem [0:NUM_NEURONS-1];
```

```
// Write port signals
reg [NEURON_ID_WIDTH-1:0] bram_wr_addr;
reg bram_we;
reg [STATE_WIDTH-1:0] bram_din;
// Read port signals
reg [NEURON_ID_WIDTH-1:0] bram_rd_addr;
reg [STATE_WIDTH-1:0] bram_dout;



// SDP BRAM: Write port (Port A)
always @(posedge clk) begin
if (bram_we)
neuron_state_mem[bram_wr_addr] <= bram_din;
end



// SDP BRAM: Read port (Port B)
always @(posedge clk) begin
bram_dout <= neuron_state_mem[bram_rd_addr];
end



// Convenience splits for read output (serves both leak and spike paths)
wire [DATA_WIDTH-1:0] mem_rd = bram_dout[STATE_WIDTH-1:REFRAC_WIDTH];
wire [REFRAC_WIDTH-1:0] ref_rd = bram_dout[REFRAC_WIDTH-1:0];



//============================================================================
// Spike Flag Memory (BRAM-based bitmap)
//============================================================================
// One bit per neuron, packed 8 per byte.
// Set from spike processing, cleared by output scan.
//============================================================================
localparam SF_DEPTH = (NUM_NEURONS + 7) / 8;
localparam SF_ADDR_W = (SF_DEPTH <= 1) ? 1 : $clog2(SF_DEPTH);



// Spike flag: distributed RAM (small, has sync reset + read-modify-write)
reg [7:0] spike_flag_mem [0:SF_DEPTH-1];



// Set interface
reg sf_set_pending;
reg [SF_ADDR_W-1:0] sf_set_addr;
reg [2:0] sf_set_bit;



// Clear interface
reg sf_clear_pending;
reg [SF_ADDR_W-1:0] sf_clear_addr;
reg [7:0] sf_clear_mask;



always @(posedge clk) begin
if (!rst_n) begin : sf_rst_blk
integer j;
for (j = 0; j < SF_DEPTH; j = j + 1)
spike_flag_mem[j] <= 8'd0;
end else begin
if (sf_set_pending)
spike_flag_mem[sf_set_addr] <= spike_flag_mem[sf_set_addr] | (8'd1 << sf_set_bit);
if (sf_clear_pending)
spike_flag_mem[sf_clear_addr] <= spike_flag_mem[sf_clear_addr] & sf_clear_mask;
end
end



//============================================================================
// Input Spike FIFO (LUT RAM - small, async read needed)
//============================================================================
localparam FIFO_WIDTH = NEURON_ID_WIDTH + WEIGHT_WIDTH + 1;
reg [FIFO_WIDTH-1:0] spike_fifo [0:SPIKE_BUFFER_DEPTH-1];
reg [$clog2(SPIKE_BUFFER_DEPTH)-1:0] fifo_wr_ptr, fifo_rd_ptr;
reg [$clog2(SPIKE_BUFFER_DEPTH):0] fifo_count;
```

```
wire fifo_empty = (fifo_count == 0);
wire fifo_full = (fifo_count == SPIKE_BUFFER_DEPTH);
assign s_axis_spike_ready = !fifo_full;



// FIFO async-read head
wire [NEURON_ID_WIDTH-1:0] fifo_dest =
spike_fifo[fifo_rd_ptr][NEURON_ID_WIDTH-1:0];
wire [WEIGHT_WIDTH-1:0] fifo_weight =
spike_fifo[fifo_rd_ptr][NEURON_ID_WIDTH+WEIGHT_WIDTH-1:NEURON_ID_WIDTH];
wire fifo_exc = spike_fifo[fifo_rd_ptr][FIFO_WIDTH-1];



//============================================================================
// Spike processing registers (declared early for DSP block usage)
//============================================================================
reg [NEURON_ID_WIDTH-1:0] sp_addr;
reg [WEIGHT_WIDTH-1:0] sp_weight;
reg sp_exc;
reg sp_fired;



//============================================================================
// Shift-based Leak Parameters
//============================================================================
wire [2:0] shift1 = global_leak_rate[2:0];
wire [4:0] shift2_cfg = global_leak_rate[7:3];
wire [2:0] shift2 = shift2_cfg[2:0];
wire shift2_en = (shift2_cfg != 5'd0);



// Leak combinational path (DSP-inferred add)
wire [DATA_WIDTH-1:0] leak_primary = (shift1 != 3'd0) ? (mem_rd >> shift1) :
{DATA_WIDTH{1'b0}};
wire [DATA_WIDTH-1:0] leak_secondary = (shift2_en && shift2 != 3'd0) ? (mem_rd >>
shift2) : {DATA_WIDTH{1'b0}};
(* use_dsp = "yes" *) wire [DATA_WIDTH-1:0] leak_total;
assign leak_total = leak_primary + leak_secondary;



//============================================================================
// DSP-friendly Synaptic Accumulation
//============================================================================
// Force DSP48E1 inference: accumulate + threshold compare
(* use_dsp = "yes" *) wire [DATA_WIDTH:0] synaptic_sum;
assign synaptic_sum = {1'b0, mem_rd} + {{(DATA_WIDTH-WEIGHT_WIDTH+1){1'b0}},
sp_weight};
(* use_dsp = "yes" *) wire [DATA_WIDTH:0] threshold_diff;
assign threshold_diff = synaptic_sum - {1'b0, global_threshold};
wire threshold_hit = sp_exc && ~threshold_diff[DATA_WIDTH]; // MSB=0 means sum >=
threshold



//============================================================================
// Main State Machine
//============================================================================
localparam [2:0]
ST_IDLE = 3'd0,
ST_LEAK_RD = 3'd1,
ST_LEAK_CMP = 3'd2,
ST_LEAK_WR = 3'd3,
ST_SPIKE_RD = 3'd4,
ST_SPIKE_CMP = 3'd5,
ST_SPIKE_WR = 3'd6;



reg [2:0] state;
reg [NEURON_ID_WIDTH-1:0] leak_idx;
reg leak_cycle_done;



// Leak hold registers
reg [NEURON_ID_WIDTH-1:0] leak_addr_hold;
```

```
assign array_busy = (state != ST_IDLE) || !fifo_empty;



always @(posedge clk) begin
if (!rst_n) begin
state <= ST_IDLE;
leak_idx <= 0;
leak_cycle_done <= 0;
bram_we <= 0;
bram_wr_addr <= 0;
bram_rd_addr <= 0;
bram_din <= 0;
sp_addr <= 0;
sp_weight <= 0;
sp_exc <= 0;
sp_fired <= 0;
leak_addr_hold <= 0;
sf_set_pending <= 0;
sf_set_addr <= 0;
sf_set_bit <= 0;
fifo_wr_ptr <= 0;
fifo_rd_ptr <= 0;
fifo_count <= 0;
end else begin
// Defaults
bram_we <= 0;
sf_set_pending <= 0;



// FIFO write (always active)
if (s_axis_spike_valid && !fifo_full) begin
spike_fifo[fifo_wr_ptr] <= {s_axis_spike_exc_inh, s_axis_spike_weight,
s_axis_spike_dest_id};
fifo_wr_ptr <= fifo_wr_ptr + 1;
end



if (enable || state != ST_IDLE) begin
case (state)
//------------------------------------------------------------------------------
ST_IDLE: begin
sp_fired <= 0;



if (!fifo_empty) begin
// Capture spike from FIFO, issue BRAM read
sp_addr <= fifo_dest;
sp_weight <= fifo_weight;
sp_exc <= fifo_exc;
bram_rd_addr <= fifo_dest;
fifo_rd_ptr <= fifo_rd_ptr + 1;
state <= ST_SPIKE_RD;
end else if (!leak_cycle_done) begin
bram_rd_addr <= leak_idx;
state <= ST_LEAK_RD;
end
end



//------------------------------------------------------------------------------
// Leak pipeline: RD -> CMP -> WR (3 cycles per neuron)
//------------------------------------------------------------------------------
ST_LEAK_RD: begin
leak_addr_hold <= bram_rd_addr;
// Interrupt leak for incoming spike
if (!fifo_empty) begin
sp_addr <= fifo_dest;
sp_weight <= fifo_weight;
sp_exc <= fifo_exc;
bram_rd_addr <= fifo_dest;
fifo_rd_ptr <= fifo_rd_ptr + 1;
state <= ST_SPIKE_RD;
end else begin
state <= ST_LEAK_CMP;
end
end



```

```
ST_LEAK_CMP: begin
// Interrupt for spike
if (!fifo_empty) begin
// Save leak result (write port), switch to spike (read port)
bram_we <= 1;
bram_wr_addr <= leak_addr_hold;
if (ref_rd > 0)
bram_din <= {{DATA_WIDTH{1'b0}}, ref_rd - 1'b1};
else if (mem_rd > leak_total)
bram_din <= {mem_rd - leak_total, {REFRAC_WIDTH{1'b0}}};
else
bram_din <= {STATE_WIDTH{1'b0}};



leak_idx <= leak_addr_hold + 1;
if (leak_addr_hold + 1 >= NUM_NEURONS) begin
leak_idx <= 0;
leak_cycle_done <= 1;
end



sp_addr <= fifo_dest;
sp_weight <= fifo_weight;
sp_exc <= fifo_exc;
bram_rd_addr <= fifo_dest;
fifo_rd_ptr <= fifo_rd_ptr + 1;
state <= ST_SPIKE_RD;
end  else  begin
state <= ST_LEAK_WR;
end
end



ST_LEAK_WR: begin
bram_we <= 1;
bram_wr_addr <= leak_addr_hold; // Write port (separate from read)



if (ref_rd > 0)
bram_din <= {{DATA_WIDTH{1'b0}}, ref_rd - 1'b1};
else if (mem_rd > leak_total)
bram_din <= {mem_rd - leak_total, {REFRAC_WIDTH{1'b0}}};
else
bram_din <= {STATE_WIDTH{1'b0}};



// Advance leak index
leak_idx <= leak_idx + 1;
if (leak_idx + 1 >= NUM_NEURONS) begin
leak_idx <= 0;
leak_cycle_done <= 1;
state <= ST_IDLE;
end else begin
bram_rd_addr <= leak_idx + 1; // Read port (independent)
state <= ST_LEAK_RD;
end
end



//-----------------------------------------------------------------------------
// Spike pipeline: RD -> CMP -> WR (3 cycles)
//-----------------------------------------------------------------------------
ST_SPIKE_RD: begin
state <= ST_SPIKE_CMP;
end



ST_SPIKE_CMP: begin
sp_fired <= 0;
if (ref_rd == 0 && sp_exc) begin
if (~threshold_diff[DATA_WIDTH]) // DSP: sum >= threshold
sp_fired <= 1;
end
state <= ST_SPIKE_WR;
end



ST_SPIKE_WR: begin
```

```
bram_we <= 1;
bram_wr_addr <= sp_addr;



if (ref_rd > 0) begin
bram_din <= bram_dout; // Keep as-is during refractory
end else if (sp_fired) begin
bram_din <= {{DATA_WIDTH{1'b0}}, global_refrac_period};
sf_set_pending <= 1;
sf_set_addr <= sp_addr[NEURON_ID_WIDTH-1:3];
sf_set_bit <= sp_addr[2:0];
end else begin
// Not fired - update membrane
if (sp_exc) begin
if (synaptic_sum[DATA_WIDTH])
bram_din <= {{DATA_WIDTH{1'b1}}, ref_rd}; // Saturate
else
bram_din <= {synaptic_sum[DATA_WIDTH-1:0], ref_rd};
end else begin
// Inhibitory
if (mem_rd >= sp_weight)
bram_din <= {mem_rd - {{(DATA_WIDTH-WEIGHT_WIDTH){1'b0}}, sp_weight}, ref_rd};
else
bram_din <= {{DATA_WIDTH{1'b0}}, ref_rd};
end
end



state <= ST_IDLE;
end



default: state <= ST_IDLE;
endcase



// Reset leak_cycle_done for continuous operation
if (leak_cycle_done && fifo_empty && state == ST_IDLE)
leak_cycle_done <= 0;



// Config write override (highest priority)
if (config_we && config_addr < NUM_NEURONS) begin
bram_we <= 1;
bram_wr_addr <= config_addr;
case (config_data[31:30])
2'b00: bram_din <= {config_data[DATA_WIDTH-1:0], bram_dout[REFRAC_WIDTH-1:0]};
2'b01: bram_din <= {bram_dout[STATE_WIDTH-1:REFRAC_WIDTH],
config_data[REFRAC_WIDTH-1:0]};
default: bram_din <= bram_dout;
endcase
end
end



// FIFO count management
// Detect transitions that consume from FIFO
if (state == ST_IDLE && !fifo_empty && enable) begin
// Will consume one entry
if (s_axis_spike_valid && !fifo_full)
fifo_count <= fifo_count; // simultaneous push+pop
else
fifo_count <= fifo_count - 1;
end else if ((state == ST_LEAK_RD || state == ST_LEAK_CMP) && !fifo_empty) begin
// Leak interrupted by spike - consume
if (s_axis_spike_valid && !fifo_full)
fifo_count <= fifo_count;
else
fifo_count <= fifo_count - 1;
end else if (s_axis_spike_valid && !fifo_full) begin
fifo_count <= fifo_count + 1;
end
end
end



//============================================================================
// Output Spike Generation
//============================================================================
```

```
reg [NEURON_ID_WIDTH-1:0] scan_idx;
reg [31:0] total_spikes;
reg [31:0] throughput_cnt;
reg [7:0] active_neuron_cnt;
reg [7:0] scan_byte;
reg [2:0] scan_bit_pos;
reg scan_active;



assign spike_count = total_spikes;
assign throughput_counter = throughput_cnt;
assign active_neurons = active_neuron_cnt;



always @(posedge clk) begin
if (!rst_n) begin
m_axis_spike_valid <= 0;
m_axis_spike_neuron_id <= 0;
scan_idx <= 0;
total_spikes <= 0;
throughput_cnt <= 0;
active_neuron_cnt <= 0;
sf_clear_pending <= 0;
sf_clear_addr <= 0;
sf_clear_mask <= 8'hFF;
scan_byte <= 0;
scan_bit_pos <= 0;
scan_active <= 0;
end else begin
sf_clear_pending <= 0;
throughput_cnt <= throughput_cnt + 1;



if (m_axis_spike_ready || !m_axis_spike_valid) begin
m_axis_spike_valid <= 0;



if (!scan_active) begin
scan_byte <= spike_flag_mem[scan_idx[NEURON_ID_WIDTH-1:3]];
scan_bit_pos <= 0;
scan_active <= 1;
end else begin
if (scan_byte[scan_bit_pos]) begin
m_axis_spike_valid <= 1;
m_axis_spike_neuron_id <= {scan_idx[NEURON_ID_WIDTH-1:3], scan_bit_pos};
total_spikes <= total_spikes + 1;
active_neuron_cnt <= active_neuron_cnt + 1;
sf_clear_pending <= 1;
sf_clear_addr <= scan_idx[NEURON_ID_WIDTH-1:3];
sf_clear_mask <= ~(8'd1 << scan_bit_pos);
end



if (scan_bit_pos == 3'd7) begin
scan_active <= 0;
if (scan_idx + 8 >= NUM_NEURONS)
scan_idx <= 0;
else
scan_idx <= scan_idx + 8;
end else begin
scan_bit_pos <= scan_bit_pos + 1;
end
end
end



if (throughput_cnt[15:0] == 0)
active_neuron_cnt <= 0;
end
end



//============================================================================
// Memory Init (simulation only)
//============================================================================
integer init_i;
initial begin
for (init_i = 0; init_i < NUM_NEURONS; init_i = init_i + 1)
neuron_state_mem[init_i] = {STATE_WIDTH{1'b0}};
```

```
for (init_i = 0; init_i < SF_DEPTH; init_i = init_i + 1)
spike_flag_mem[init_i] = 8'd0;
for (init_i = 0; init_i < SPIKE_BUFFER_DEPTH; init_i = init_i + 1)
spike_fifo[init_i] = {FIFO_WIDTH{1'b0}};
end



endmodule
```

10. SNN Accelerator — Top Level

Top-level integration of the neuron array, AXI4 control/data interfaces, and spike-buffering logic into a single accelerator core.

Module: snn_accelerator_top

Top-level SNN accelerator core (neuron array + AXI interfaces + control).

```
module snn_accelerator_top #(
parameter C_S_AXI_DATA_WIDTH = 32,
parameter C_S_AXI_ADDR_WIDTH = 32,
parameter C_AXIS_DATA_WIDTH = 32,
parameter NUM_NEURONS = 720,
parameter NUM_AXONS = 1024,
parameter NUM_PARALLEL_UNITS = 8,
parameter SPIKE_BUFFER_DEPTH = 64,
parameter NEURON_ID_WIDTH = $clog2(NUM_NEURONS),
parameter AXON_ID_WIDTH = 10,
parameter DATA_WIDTH = 16,
parameter WEIGHT_WIDTH = 8,
parameter LEAK_WIDTH = 8,
parameter THRESHOLD_WIDTH = 16,
parameter REFRAC_WIDTH = 8,
parameter ROUTER_BUFFER_DEPTH = 512,
parameter USE_BRAM = 1,
parameter USE_DSP = 1
)(
input wire aclk,
input wire aresetn,



// AXI4-Lite Slave Interface (Configuration)
input wire [C_S_AXI_ADDR_WIDTH-1:0] s_axi_awaddr,
input wire [2:0] s_axi_awprot,
input wire s_axi_awvalid,
output wire s_axi_awready,
input wire [C_S_AXI_DATA_WIDTH-1:0] s_axi_wdata,
input wire [(C_S_AXI_DATA_WIDTH/8)-1:0] s_axi_wstrb,
input wire s_axi_wvalid,
output wire s_axi_wready,
output wire [1:0] s_axi_bresp,
output wire s_axi_bvalid,
input wire s_axi_bready,
input wire [C_S_AXI_ADDR_WIDTH-1:0] s_axi_araddr,
input wire [2:0] s_axi_arprot,
input wire s_axi_arvalid,
output wire s_axi_arready,
output wire [C_S_AXI_DATA_WIDTH-1:0] s_axi_rdata,
output wire [1:0] s_axi_rresp,
output wire s_axi_rvalid,
input wire s_axi_rready,



// AXI4-Stream Slave Interface (Input Spikes from PS)
input wire [C_AXIS_DATA_WIDTH-1:0] s_axis_tdata,
input wire s_axis_tvalid,
output wire s_axis_tready,
input wire [3:0] s_axis_tkeep,
input wire [3:0] s_axis_tstrb,
input wire s_axis_tuser,
input wire s_axis_tlast,
input wire s_axis_tid,
input wire s_axis_tdest,



// AXI4-Stream Master Interface (Output Spikes to PS)
output wire [C_AXIS_DATA_WIDTH-1:0] m_axis_tdata,
output wire m_axis_tvalid,
input wire m_axis_tready,
output wire [3:0] m_axis_tkeep,
output wire [3:0] m_axis_tstrb,
output wire m_axis_tuser,
output wire m_axis_tlast,
output wire m_axis_tid,
output wire m_axis_tdest,



// Interrupt to PS
```

```
output wire interrupt,
// Board I/O (Optional)
output wire [3:0] led,
input wire [1:0] sw,
input wire [3:0] btn,
output wire led4_r,
output wire led4_g,
output wire led4_b,
output wire led5_r,
output wire led5_g,
output wire led5_b
);



// Reuse the HLS-based top that already encapsulates all AXI handling.
snn_accelerator_hls_top #(
.C_S_AXI_DATA_WIDTH(C_S_AXI_DATA_WIDTH),
.C_S_AXI_ADDR_WIDTH(7),
.C_AXIS_DATA_WIDTH(C_AXIS_DATA_WIDTH),
.NUM_NEURONS(NUM_NEURONS),
.NUM_AXONS(NUM_AXONS),
.NUM_PARALLEL_UNITS(NUM_PARALLEL_UNITS),
.SPIKE_BUFFER_DEPTH(SPIKE_BUFFER_DEPTH),
.NEURON_ID_WIDTH(NEURON_ID_WIDTH),
.AXON_ID_WIDTH(AXON_ID_WIDTH),
.DATA_WIDTH(DATA_WIDTH),
.WEIGHT_WIDTH(WEIGHT_WIDTH),
.LEAK_WIDTH(LEAK_WIDTH),
.THRESHOLD_WIDTH(THRESHOLD_WIDTH),
.REFRAC_WIDTH(REFRAC_WIDTH),
.ROUTER_BUFFER_DEPTH(ROUTER_BUFFER_DEPTH),
.USE_BRAM(USE_BRAM),
.USE_DSP(USE_DSP)
) u_hls_top (
.aclk(aclk),
.aresetn(aresetn),



// AXI4-Lite
.s_axi_awaddr(s_axi_awaddr[6:0]),
.s_axi_awvalid(s_axi_awvalid),
.s_axi_awready(s_axi_awready),
.s_axi_wdata(s_axi_wdata),
.s_axi_wstrb(s_axi_wstrb),
.s_axi_wvalid(s_axi_wvalid),
.s_axi_wready(s_axi_wready),
.s_axi_bresp(s_axi_bresp),
.s_axi_bvalid(s_axi_bvalid),
.s_axi_bready(s_axi_bready),
.s_axi_araddr(s_axi_araddr[6:0]),
.s_axi_arvalid(s_axi_arvalid),
.s_axi_arready(s_axi_arready),
.s_axi_rdata(s_axi_rdata),
.s_axi_rresp(s_axi_rresp),
.s_axi_rvalid(s_axi_rvalid),
.s_axi_rready(s_axi_rready),



// AXI4-Stream
.s_axis_tdata(s_axis_tdata),
.s_axis_tvalid(s_axis_tvalid),
.s_axis_tready(s_axis_tready),
.s_axis_tkeep(s_axis_tkeep),
.s_axis_tstrb(s_axis_tstrb),
.s_axis_tuser(s_axis_tuser),
.s_axis_tlast(s_axis_tlast),
.s_axis_tid(s_axis_tid),
.s_axis_tdest(s_axis_tdest),
.m_axis_tdata(m_axis_tdata),
.m_axis_tvalid(m_axis_tvalid),
.m_axis_tready(m_axis_tready),
.m_axis_tkeep(m_axis_tkeep),
.m_axis_tstrb(m_axis_tstrb),
.m_axis_tuser(m_axis_tuser),
.m_axis_tlast(m_axis_tlast),
.m_axis_tid(m_axis_tid),
.m_axis_tdest(m_axis_tdest),



// Interrupt and board I/O
.interrupt(interrupt),
.led(led),
```

```
.sw(sw),
.btn(btn),
.led4_r(led4_r),
.led4_g(led4_g),
.led4_b(led4_b),
.led5_r(led5_r),
.led5_g(led5_g),
.led5_b(led5_b)
);



endmodule
```

11. SNN System Integration — Top Level

System-level wrapper that integrates the accelerator core with HLS-generated neuron and weight-memory blocks, scaling network parameters for full deployment.

Module: snn_integrated_top

Full system integration of accelerator core with HLS-generated memories.

```
module snn_integrated_top #(
// System Parameters
parameter NUM_NEURONS = 1024, // Scaled up: BRAM-backed, LUT-neutral
parameter NUM_AXONS = 1024,
parameter NUM_PARALLEL_UNITS = 4,
parameter SPIKE_BUFFER_DEPTH = 64,
parameter HLS_NEURON_ID_WIDTH = 10, // Matches HLS neuron_id_t (10-bit)
parameter HLS_MAX_NEURONS = 720, // HLS weight_memory covers neurons 0..719
parameter NEURON_ID_WIDTH = (NUM_NEURONS <= 256) ? 8 :
(NUM_NEURONS <= 512) ? 9 : 10,
parameter AXON_ID_WIDTH = 10,
parameter DATA_WIDTH = 16,
parameter WEIGHT_WIDTH = 8,
parameter LEAK_WIDTH = 8,
parameter THRESHOLD_WIDTH = 16,
parameter REFRAC_WIDTH = 8,
parameter ROUTER_BUFFER_DEPTH = 512
)(
//------------------------------------------------------------------------------------------------------------------------
// DDR Interface (directly from PS)
//------------------------------------------------------------------------------------------------------------------------
inout wire [14:0] DDR_addr,
inout wire [2:0] DDR_ba,
inout wire DDR_cas_n,
inout wire DDR_ck_n,
inout wire DDR_ck_p,
inout wire DDR_cke,
inout wire DDR_cs_n,
inout wire [3:0] DDR_dm,
inout wire [31:0] DDR_dq,
inout wire [3:0] DDR_dqs_n,
inout wire [3:0] DDR_dqs_p,
inout wire DDR_odt,
inout wire DDR_ras_n,
inout wire DDR_reset_n,
inout wire DDR_we_n,



//------------------------------------------------------------------------------------------------------------------------
// Fixed IO (PS)
//------------------------------------------------------------------------------------------------------------------------
inout wire FIXED_IO_ddr_vrn,
inout wire FIXED_IO_ddr_vrp,
inout wire [53:0] FIXED_IO_mio,
inout wire FIXED_IO_ps_clk,
inout wire FIXED_IO_ps_porb,
inout wire FIXED_IO_ps_srstb
);



//================================================================================
// Internal Signals: Clock and Reset
//================================================================================
wire clk_100mhz;
wire rst_n_sync;
wire debug_learning_active;



//================================================================================
// HLS <-> RTL Interface Signals (from Block Design)
//================================================================================
// HLS → RTL: Spikes from HLS to RTL neurons (8-bit interface)
wire hls_spike_out_valid;
wire [HLS_NEURON_ID_WIDTH-1:0] hls_spike_out_neuron_id;
wire [WEIGHT_WIDTH-1:0] hls_spike_out_weight;
wire rtl_spike_in_ready;



// RTL → HLS: Spikes from RTL neurons to HLS (8-bit interface)
wire rtl_spike_out_valid;
```

```
wire [HLS_NEURON_ID_WIDTH-1:0] rtl_spike_out_neuron_id;
wire [WEIGHT_WIDTH-1:0] rtl_spike_out_weight;
wire hls_spike_in_ready;



//============================================================================
// HLS ↔ RTL Width Adapters (8-bit HLS ↔ 10-bit internal)
// HLS addresses neurons 0-255; neurons 256-1023 are RTL-internal
//============================================================================
wire [NEURON_ID_WIDTH-1:0] hls_spike_id_extended;
assign hls_spike_id_extended =
{{(NEURON_ID_WIDTH-HLS_NEURON_ID_WIDTH){1'b0}},
hls_spike_out_neuron_id};



// SNN Control
wire hls_snn_enable;
wire hls_snn_reset;
wire rtl_snn_ready;
wire rtl_snn_busy;



// HLS neuron parameter outputs (monitoring only)
wire [15:0] hls_threshold_out;
wire [15:0] hls_leak_rate_out;



//============================================================================
// Config Register Interface (from AXI-Lite slave in Block Design)
//============================================================================
wire cfg_router_config_we;
wire [31:0] cfg_router_config_addr;
wire [31:0] cfg_router_config_wdata;
wire [31:0] cfg_router_config_rdata;
wire cfg_neuron_config_we;
wire [9:0] cfg_neuron_config_addr;
wire [31:0] cfg_neuron_config_wdata;



wire [15:0] cfg_global_threshold;
wire [7:0] cfg_global_leak_rate;
wire [7:0] cfg_global_refrac_period;



//============================================================================
// Internal Spike Router Signals
//============================================================================
wire router_spike_valid;
wire [NEURON_ID_WIDTH-1:0] router_spike_dest_id;
wire [WEIGHT_WIDTH-1:0] router_spike_weight;
wire router_spike_exc_inh;
wire router_spike_ready;



// Router input selection (HLS or recurrent)
wire router_input_valid;
wire [NEURON_ID_WIDTH-1:0] router_input_neuron_id;
wire router_input_ready;



//============================================================================
// Neuron Array Output Signals
//============================================================================
wire neuron_spike_valid;
wire [NEURON_ID_WIDTH-1:0] neuron_spike_id;
wire neuron_spike_ready;



// Statistics & Monitoring
wire [31:0] router_spike_count;
wire [31:0] neuron_spike_count;
wire router_busy;
wire neuron_array_busy;
wire fifo_overflow;
wire [31:0] throughput_counter;
wire [7:0] active_neurons;
//============================================================================
```

```
// Block Design Instantiation (PS + HLS IP + AXI + Config Regs)
//============================================================================



design_1_wrapper u_block_design (
// DDR Interface
.DDR_addr (DDR_addr),
.DDR_ba (DDR_ba),
.DDR_cas_n (DDR_cas_n),
.DDR_ck_n (DDR_ck_n),
.DDR_ck_p (DDR_ck_p),
.DDR_cke (DDR_cke),
.DDR_cs_n (DDR_cs_n),
.DDR_dm (DDR_dm),
.DDR_dq (DDR_dq),
.DDR_dqs_n (DDR_dqs_n),
.DDR_dqs_p (DDR_dqs_p),
.DDR_odt (DDR_odt),
.DDR_ras_n (DDR_ras_n),
.DDR_reset_n (DDR_reset_n),
.DDR_we_n (DDR_we_n),



// Fixed IO
.FIXED_IO_ddr_vrn (FIXED_IO_ddr_vrn),
.FIXED_IO_ddr_vrp (FIXED_IO_ddr_vrp),
.FIXED_IO_mio (FIXED_IO_mio),
.FIXED_IO_ps_clk (FIXED_IO_ps_clk),
.FIXED_IO_ps_porb (FIXED_IO_ps_porb),
.FIXED_IO_ps_srstb (FIXED_IO_ps_srstb),



// PL Clock/Reset outputs
.clk_100mhz (clk_100mhz),
.rst_n_sync (rst_n_sync),



// Debug
.debug_learning_active (debug_learning_active),



//------------------------------------------------------------------------------------------------------------
// HLS → RTL Spike Interface
//------------------------------------------------------------------------------------------------------------
.hls_spike_out_valid (hls_spike_out_valid),
.hls_spike_out_neuron_id (hls_spike_out_neuron_id),
.hls_spike_out_weight (hls_spike_out_weight),
.rtl_spike_in_ready (rtl_spike_in_ready),



//------------------------------------------------------------------------------------------------------------
// RTL → HLS Spike Interface (for STDP learning)
//------------------------------------------------------------------------------------------------------------
.rtl_spike_out_valid (rtl_spike_out_valid),
.rtl_spike_out_neuron_id (rtl_spike_out_neuron_id),
.rtl_spike_out_weight (rtl_spike_out_weight),
.hls_spike_in_ready (hls_spike_in_ready),



//------------------------------------------------------------------------------------------------------------
// SNN Control Interface
//------------------------------------------------------------------------------------------------------------
.hls_snn_enable (hls_snn_enable),
.hls_snn_reset (hls_snn_reset),
.rtl_snn_ready (rtl_snn_ready),
.rtl_snn_busy (rtl_snn_busy),



//------------------------------------------------------------------------------------------------------------
// HLS Neuron Parameter Outputs (monitoring)
//------------------------------------------------------------------------------------------------------------
.hls_threshold_out (hls_threshold_out),
.hls_leak_rate_out (hls_leak_rate_out),



//------------------------------------------------------------------------------------------------------------
// Config Register Interface
```

```
//-----------------------------------------------------------------------------------------------------------
// Router config
.cfg_router_config_we (cfg_router_config_we),
.cfg_router_config_addr (cfg_router_config_addr),
.cfg_router_config_wdata (cfg_router_config_wdata),
.cfg_router_config_rdata (cfg_router_config_rdata),
// Neuron config
.cfg_neuron_config_we (cfg_neuron_config_we),
.cfg_neuron_config_addr (cfg_neuron_config_addr),
.cfg_neuron_config_wdata (cfg_neuron_config_wdata),



// Global parameters
.cfg_global_threshold (cfg_global_threshold),
.cfg_global_leak_rate (cfg_global_leak_rate),
.cfg_global_refrac_period(cfg_global_refrac_period),



// Status feedback
.cfg_router_spike_count (router_spike_count),
.cfg_neuron_spike_count (neuron_spike_count),
.cfg_fifo_overflow (fifo_overflow),
.cfg_active_neurons (active_neurons),
.cfg_throughput_counter (throughput_counter)
);



//============================================================================
// Spike Input Multiplexer
// Priority: HLS spikes > Recurrent neuron spikes
//============================================================================



// Simple priority: HLS has priority when sending spikes
assign router_input_valid = hls_spike_out_valid | neuron_spike_valid;
assign router_input_neuron_id = hls_spike_out_valid ? hls_spike_id_extended :
neuron_spike_id;



// Ready signals
assign rtl_spike_in_ready = router_input_ready;
assign neuron_spike_ready = router_input_ready & ~hls_spike_out_valid;



//============================================================================
// RTL → HLS Connection
// Send neuron output spikes to HLS for STDP learning
// GUARD: Only forward spikes from neurons 0-255 to avoid ID aliasing
// Neurons 256-1023 participate in recurrent activity but are not
// reported to HLS (8-bit truncation would cause wrong neuron IDs)
//============================================================================



wire neuron_in_hls_range = (neuron_spike_id < HLS_MAX_NEURONS);



assign rtl_spike_out_valid = neuron_spike_valid & neuron_in_hls_range;
assign rtl_spike_out_neuron_id = neuron_spike_id[HLS_NEURON_ID_WIDTH-1:0];
assign rtl_spike_out_weight = router_spike_weight; // Use current weight



//============================================================================
// SNN Status to HLS
//============================================================================



assign rtl_snn_ready = ~router_busy & ~neuron_array_busy;
assign rtl_snn_busy = router_busy | neuron_array_busy;



//============================================================================
// Verilog RTL: Spike Router (AER-based)
//============================================================================
```

```
spike_router #(
.NUM_NEURONS (NUM_NEURONS),
.MAX_FANOUT (32),
.WEIGHT_WIDTH (WEIGHT_WIDTH),
.NEURON_ID_WIDTH (NEURON_ID_WIDTH),
.DELAY_WIDTH (8),
.FIFO_DEPTH (ROUTER_BUFFER_DEPTH)
) u_spike_router (
.clk (clk_100mhz),
.rst_n (rst_n_sync & ~hls_snn_reset),



// Input (from HLS or recurrent)
.s_spike_valid (router_input_valid),
.s_spike_neuron_id (router_input_neuron_id),
.s_spike_ready (router_input_ready),



// Output to neuron array
.m_spike_valid (router_spike_valid),
.m_spike_dest_id (router_spike_dest_id),
.m_spike_weight (router_spike_weight),
.m_spike_exc_inh (router_spike_exc_inh),
.m_spike_ready (router_spike_ready),



// Configuration (driven by AXI-Lite config registers)
.config_we (cfg_router_config_we),
.config_addr (cfg_router_config_addr),
.config_data (cfg_router_config_wdata),
.config_readdata (cfg_router_config_rdata),



// Statistics
.routed_spike_count (router_spike_count),
.router_busy (router_busy),
.fifo_overflow (fifo_overflow)
);



//============================================================================
// Verilog RTL: LIF Neuron Array (AC-based, energy-efficient)
//============================================================================



lif_neuron_array #(
.NUM_NEURONS (NUM_NEURONS),
.NUM_AXONS (NUM_AXONS),
.DATA_WIDTH (DATA_WIDTH),
.WEIGHT_WIDTH (WEIGHT_WIDTH),
.THRESHOLD_WIDTH (THRESHOLD_WIDTH),
.LEAK_WIDTH (LEAK_WIDTH),
.REFRAC_WIDTH (REFRAC_WIDTH),
.NUM_PARALLEL_UNITS (NUM_PARALLEL_UNITS),
.SPIKE_BUFFER_DEPTH (SPIKE_BUFFER_DEPTH),
.USE_BRAM (1),
.USE_DSP (1) // DSP48E1 for synaptic accumulation
) u_neuron_array (
.clk (clk_100mhz),
.rst_n (rst_n_sync & ~hls_snn_reset),
.enable (hls_snn_enable),



// Input from spike router
.s_axis_spike_valid (router_spike_valid),
.s_axis_spike_dest_id (router_spike_dest_id),
.s_axis_spike_weight (router_spike_weight),
.s_axis_spike_exc_inh (router_spike_exc_inh),
.s_axis_spike_ready (router_spike_ready),



// Output spikes (to HLS for learning + recurrent)
.m_axis_spike_valid (neuron_spike_valid),
.m_axis_spike_neuron_id (neuron_spike_id),
.m_axis_spike_ready (hls_spike_in_ready), // HLS controls flow
```

```
// Configuration (driven by AXI-Lite config registers)
.config_we (cfg_neuron_config_we),
.config_addr (cfg_neuron_config_addr),
.config_data (cfg_neuron_config_wdata),



// Global neuron parameters (driven by AXI-Lite config registers)
.global_threshold (cfg_global_threshold),
.global_leak_rate (cfg_global_leak_rate),
.global_refrac_period (cfg_global_refrac_period),



// Monitoring (fed back to config register status inputs)
.spike_count (neuron_spike_count),
.array_busy (neuron_array_busy),
.throughput_counter (throughput_counter),
.active_neurons (active_neurons)
);



endmodule
```

12. PYNQ-Z2 Programmable-Logic Wrapper

Top-level programmable-logic wrapper for the PYNQ-Z2 development board, connecting on-board clocks, buttons, switches, and LEDs to the SNN accelerator for hardware demonstration.

Module: snn_pl_wrapper

PYNQ-Z2 board-level top wrapper connecting I/O to the accelerator.

```
module snn_pl_wrapper (
//-----------------------------------------------------------------------------------------------------------------------
// Clock and Reset from PYNQ-Z2 125MHz oscillator
//-----------------------------------------------------------------------------------------------------------------------
input wire sysclk, // 125 MHz system clock
//-----------------------------------------------------------------------------------------------------------------------
// Board I/O - Directly available on PYNQ-Z2
//-----------------------------------------------------------------------------------------------------------------------
output wire [3:0] led, // Regular LEDs
input wire [1:0] sw, // Switches
input wire [3:0] btn, // Push buttons



// RGB LEDs
output wire led4_r,
output wire led4_g,
output wire led4_b,
output wire led5_r,
output wire led5_g,
output wire led5_b
);



//-----------------------------------------------------------------------------------------------------------------------
// Internal Signals
//-----------------------------------------------------------------------------------------------------------------------
wire clk_100mhz;
wire locked;
wire sys_rst_n;



// AXI-Lite signals (directly connected to internal test logic)
wire [31:0] s_axi_awaddr;
wire [2:0] s_axi_awprot;
wire s_axi_awvalid;
wire s_axi_awready;
wire [31:0] s_axi_wdata;
wire [3:0] s_axi_wstrb;
wire s_axi_wvalid;
wire s_axi_wready;
wire [1:0] s_axi_bresp;
wire s_axi_bvalid;
wire s_axi_bready;
wire [31:0] s_axi_araddr;
wire [2:0] s_axi_arprot;
wire s_axi_arvalid;
wire s_axi_arready;
wire [31:0] s_axi_rdata;
wire [1:0] s_axi_rresp;
wire s_axi_rvalid;
wire s_axi_rready;



// AXI-Stream signals
wire [31:0] s_axis_tdata;
wire s_axis_tvalid;
wire s_axis_tready;
wire [3:0] s_axis_tkeep;
wire [3:0] s_axis_tstrb;
wire s_axis_tuser;
wire s_axis_tlast;
wire s_axis_tid;
wire s_axis_tdest;
wire [31:0] m_axis_tdata;
wire m_axis_tvalid;
wire m_axis_tready;
wire [3:0] m_axis_tkeep;
wire [3:0] m_axis_tstrb;
wire m_axis_tuser;
wire m_axis_tlast;
```

```
wire m_axis_tid;
wire m_axis_tdest;



wire interrupt;



//----------------------------------------------------------------------------------------------------------------
// Clock Generation
//----------------------------------------------------------------------------------------------------------------
// Using 125MHz sysclk directly from PYNQ-Z2 oscillator
// For production use requiring exact 100MHz, instantiate MMCM/PLL:
// clk_wiz_0 clk_gen (.clk_in1(sysclk), .clk_out1(clk_100mhz), .locked(locked));
assign clk_100mhz = sysclk;
assign locked = 1'b1;



//----------------------------------------------------------------------------------------------------------------
// Reset Synchronizer
//----------------------------------------------------------------------------------------------------------------
reg [3:0] rst_sync;



always @(posedge clk_100mhz or negedge btn[0]) begin
if (!btn[0]) // Use button 0 as reset
rst_sync <= 4'b0;
else
rst_sync <= {rst_sync[2:0], 1'b1};
end



assign sys_rst_n = rst_sync[3] & locked;



//----------------------------------------------------------------------------------------------------------------
// Simple AXI-Lite Test Controller
// Performs basic register writes to configure the SNN and test operation
//----------------------------------------------------------------------------------------------------------------
reg [3:0] test_state;
reg [31:0] test_addr;
reg [31:0] test_data;
reg test_write;
reg test_read;
reg [31:0] read_data;
reg [31:0] test_counter;



localparam ST_IDLE = 4'd0;
localparam ST_WRITE_CMD = 4'd1;
localparam ST_WRITE_ADDR = 4'd2;
localparam ST_WRITE_DATA = 4'd3;
localparam ST_WRITE_RESP = 4'd4;
localparam ST_READ_CMD = 4'd5;
localparam ST_READ_ADDR = 4'd6;
localparam ST_READ_DATA = 4'd7;
localparam ST_DONE = 4'd8;
localparam ST_RUN = 4'd9;



// AXI-Lite Master signals
reg [31:0] axi_awaddr_r;
reg axi_awvalid_r;
reg [31:0] axi_wdata_r;
reg axi_wvalid_r;
reg axi_bready_r;
reg [31:0] axi_araddr_r;
reg axi_arvalid_r;
reg axi_rready_r;



assign s_axi_awaddr = axi_awaddr_r;
assign s_axi_awprot = 3'b000;
assign s_axi_awvalid = axi_awvalid_r;
assign s_axi_wdata = axi_wdata_r;
assign s_axi_wstrb = 4'hF;
assign s_axi_wvalid = axi_wvalid_r;
assign s_axi_bready = axi_bready_r;
```

```
assign s_axi_araddr = axi_araddr_r;
assign s_axi_arprot = 3'b000;
assign s_axi_arvalid = axi_arvalid_r;
assign s_axi_rready = axi_rready_r;



// Register addresses (matching axi_lite_regs_v1_0)
localparam ADDR_CTRL = 32'h00; // Control register
localparam ADDR_CONFIG = 32'h04; // Config register
localparam ADDR_LEAK = 32'h08; // Leak rate
localparam ADDR_THRESHOLD = 32'h0C; // Threshold
localparam ADDR_STATUS = 32'h10; // Status (read-only)
localparam ADDR_SPIKE_CNT = 32'h14; // Spike count (read-only)



always @(posedge clk_100mhz or negedge sys_rst_n) begin
if (!sys_rst_n) begin
test_state <= ST_IDLE;
axi_awaddr_r <= 32'h0;
axi_awvalid_r <= 1'b0;
axi_wdata_r <= 32'h0;
axi_wvalid_r <= 1'b0;
axi_bready_r <= 1'b0;
axi_araddr_r <= 32'h0;
axi_arvalid_r <= 1'b0;
axi_rready_r <= 1'b0;
test_counter <= 32'h0;
read_data <= 32'h0;
end else begin
case (test_state)
ST_IDLE: begin
test_counter <= test_counter + 1'b1;
// Wait for stabilization, then configure
if (test_counter == 32'h00100000) begin
// Write threshold = 1000
axi_awaddr_r <= ADDR_THRESHOLD;
axi_wdata_r <= 32'd1000;
test_state <= ST_WRITE_ADDR;
end
end



ST_WRITE_ADDR: begin
axi_awvalid_r <= 1'b1;
if (s_axi_awready) begin
axi_awvalid_r <= 1'b0;
test_state <= ST_WRITE_DATA;
end
end



ST_WRITE_DATA: begin
axi_wvalid_r <= 1'b1;
if (s_axi_wready) begin
axi_wvalid_r <= 1'b0;
axi_bready_r <= 1'b1;
test_state <= ST_WRITE_RESP;
end
end



ST_WRITE_RESP: begin
if (s_axi_bvalid) begin
axi_bready_r <= 1'b0;
// Write control register to enable
if (axi_awaddr_r == ADDR_THRESHOLD) begin
axi_awaddr_r <= ADDR_CTRL;
axi_wdata_r <= 32'h00000001; // Enable SNN
test_state <= ST_WRITE_ADDR;
end else begin
test_state <= ST_RUN;
end
end
end



ST_RUN: begin
// SNN is running, periodically read status
test_counter <= test_counter + 1'b1;
if (test_counter[23:0] == 24'hFFFFFF) begin
```

```
axi_araddr_r <= ADDR_SPIKE_CNT;
test_state <= ST_READ_ADDR;
end
end



ST_READ_ADDR: begin
axi_arvalid_r <= 1'b1;
if (s_axi_arready) begin
axi_arvalid_r <= 1'b0;
axi_rready_r <= 1'b1;
test_state <= ST_READ_DATA;
end
end



ST_READ_DATA: begin
if (s_axi_rvalid) begin
read_data <= s_axi_rdata;
axi_rready_r <= 1'b0;
test_state <= ST_RUN;
end
end



default: test_state <= ST_IDLE;
endcase
end
end



// Expose read_data for debug via LED blinking pattern
wire [31:0] debug_read_data = read_data; // Can be probed via ILA



//----------------------------------------------------------------------------------------------------------------------
// Simple Input Spike Generator (for testing)
//----------------------------------------------------------------------------------------------------------------------
reg [31:0] spike_gen_counter;
reg spike_gen_valid;
reg [31:0] spike_gen_data;
reg [9:0] spike_neuron_id;



always @(posedge clk_100mhz or negedge sys_rst_n) begin
if (!sys_rst_n) begin
spike_gen_counter <= 32'h0;
spike_gen_valid <= 1'b0;
spike_gen_data <= 32'h0;
spike_neuron_id <= 8'h0;
end else begin
spike_gen_counter <= spike_gen_counter + 1'b1;



// Generate spike every ~1M cycles when sw[1] is on
if (sw[1] && spike_gen_counter[19:0] == 20'hFFFFF) begin
spike_gen_valid <= 1'b1;
spike_neuron_id <= spike_neuron_id + 1'b1;
// Format: [31:18]=timestamp(14b), [17:10]=weight(8b), [9:0]=neuron_id(10b)
spike_gen_data <= {spike_gen_counter[27:14], 8'd50, spike_neuron_id};
end else if (s_axis_tready) begin
spike_gen_valid <= 1'b0;
end
end
end



assign s_axis_tdata = spike_gen_data;
assign s_axis_tvalid = spike_gen_valid;
assign s_axis_tkeep = 4'hF;
assign s_axis_tstrb = 4'hF;
assign s_axis_tuser = 1'b0;
assign s_axis_tlast = spike_gen_valid; // Each spike is a complete packet
assign s_axis_tid = 1'b0;
assign s_axis_tdest = 1'b0;
```

```
// Output stream - just accept
assign m_axis_tready = 1'b1;



//-----------------------------------------------------------------------------------------------------------------------
// SNN Accelerator Instance
//-----------------------------------------------------------------------------------------------------------------------
snn_accelerator_hls_top #(
.C_S_AXI_DATA_WIDTH(32),
.C_S_AXI_ADDR_WIDTH(7),
.C_AXIS_DATA_WIDTH(32),
.NUM_NEURONS(256),
.NUM_AXONS(1024),
.NUM_PARALLEL_UNITS(8)
) u_snn_accelerator (
// Clock and Reset
.aclk (clk_100mhz),
.aresetn (sys_rst_n),



// AXI4-Lite Slave
.s_axi_awaddr (s_axi_awaddr),
.s_axi_awprot (s_axi_awprot),
.s_axi_awvalid (s_axi_awvalid),
.s_axi_awready (s_axi_awready),
.s_axi_wdata (s_axi_wdata),
.s_axi_wstrb (s_axi_wstrb),
.s_axi_wvalid (s_axi_wvalid),
.s_axi_wready (s_axi_wready),
.s_axi_bresp (s_axi_bresp),
.s_axi_bvalid (s_axi_bvalid),
.s_axi_bready (s_axi_bready),
.s_axi_araddr (s_axi_araddr),
.s_axi_arprot (s_axi_arprot),
.s_axi_arvalid (s_axi_arvalid),
.s_axi_arready (s_axi_arready),
.s_axi_rdata (s_axi_rdata),
.s_axi_rresp (s_axi_rresp),
.s_axi_rvalid (s_axi_rvalid),
.s_axi_rready (s_axi_rready),



// AXI4-Stream Slave (Input Spikes)
.s_axis_tdata (s_axis_tdata),
.s_axis_tvalid (s_axis_tvalid),
.s_axis_tready (s_axis_tready),
.s_axis_tkeep (s_axis_tkeep),
.s_axis_tstrb (s_axis_tstrb),
.s_axis_tuser (s_axis_tuser),
.s_axis_tlast (s_axis_tlast),
.s_axis_tid (s_axis_tid),
.s_axis_tdest (s_axis_tdest),



// AXI4-Stream Master (Output Spikes)
.m_axis_tdata (m_axis_tdata),
.m_axis_tvalid (m_axis_tvalid),
.m_axis_tready (m_axis_tready),
.m_axis_tkeep (m_axis_tkeep),
.m_axis_tstrb (m_axis_tstrb),
.m_axis_tuser (m_axis_tuser),
.m_axis_tlast (m_axis_tlast),
.m_axis_tid (m_axis_tid),
.m_axis_tdest (m_axis_tdest),



// Interrupt
.interrupt (interrupt),



// Board I/O
.led (led),
.sw (sw),
.btn (btn),
.led4_r (led4_r),
.led4_g (led4_g),
.led4_b (led4_b),
.led5_r (led5_r),
.led5_g (led5_g),
.led5_b (led5_b)
```

```
);


endmodule
```